

Introducción a las redes neuronales: sus orígenes y los fundamentos del aprendizaje profundo

Diego Milone



IA@Litoral

sinc(i)



Instituto de investigación en
señales e inteligencia computacional

Departamento de Informática
FICH - Universidad Nacional del Litoral
Consejo Nacional de Investigaciones Científicas y Técnicas

28 de noviembre de 2019

Organización

Redes neuronales en inteligencia artificial

Una neurona

Varias neuronas

Muuuuuuuuuuuchas neuronas

Inteligencia Natural vs Inteligencia Artificial

- A la hora de resolver problemas...
 - ¿Qué nos diferencia de una computadora?
 - ¿En qué situaciones el ser humano es mucho mejor?
 - ¿Qué tipo problemas resuelve mejor una computadora?
- Algunas capacidades del ser humano:
 - Aprendizaje,
 - Generalización,
 - Adaptabilidad,
 - Experiencia,
 - Razonamiento,
 - Creatividad,
 - Emociones, ...?

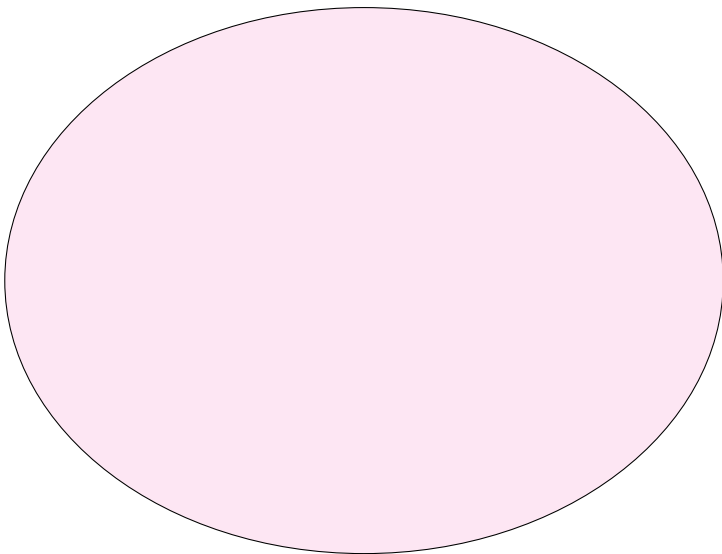
Inteligencia Natural vs Inteligencia Artificial

- A la hora de resolver problemas...
 - ¿Qué nos diferencia de una computadora?
 - ¿En qué situaciones el ser humano es mucho mejor?
 - ¿Qué tipo problemas resuelve mejor una computadora?
- Algunas capacidades del ser humano:
 - Aprendizaje,
 - Generalización,
 - Adaptabilidad,
 - Experiencia,
 - Razonamiento,
 - Creatividad,
 - Emociones, ...?
- ¿Qué fin busca la Inteligencia Artificial?

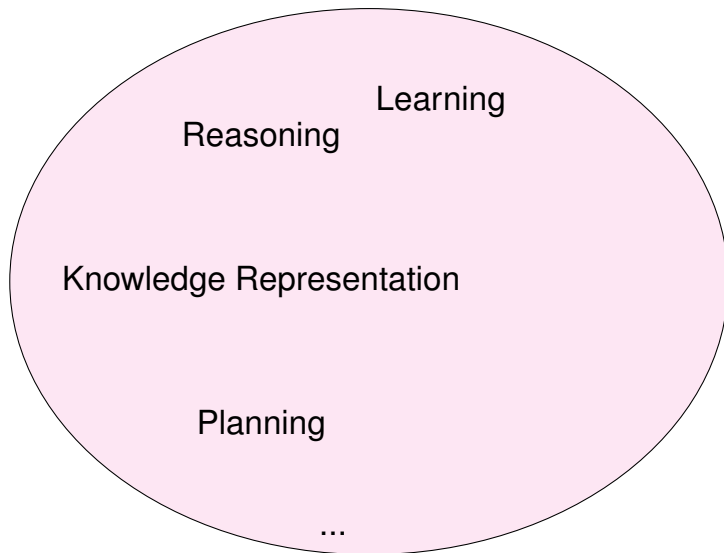
Enfoques de la Inteligencia Artificial

- ¿Qué modelamos?
 - Modelos del contenido
 - Modelos del continente
 - ...?

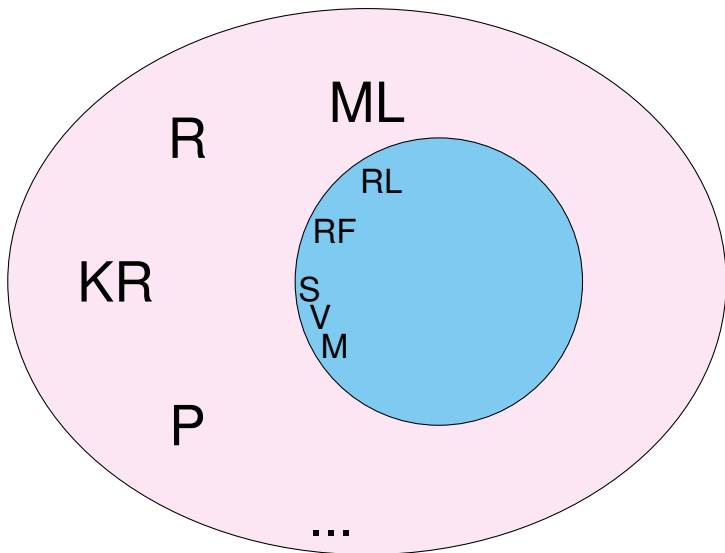
Inteligencia Artificial



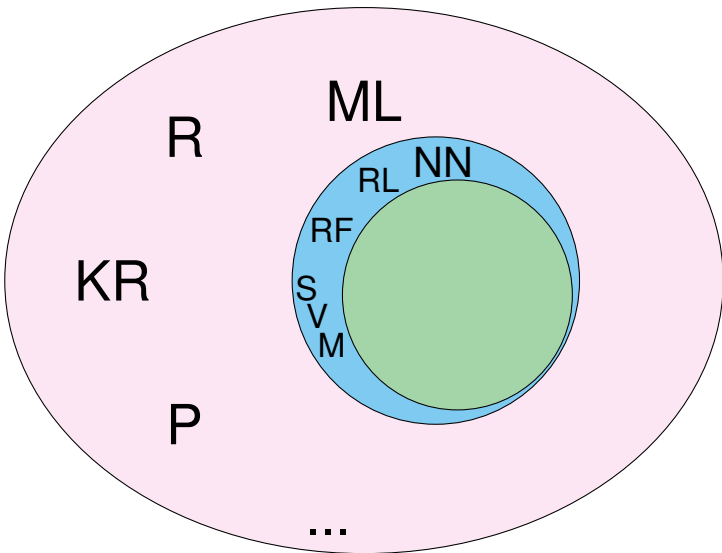
Inteligencia Artificial



Inteligencia Artificial

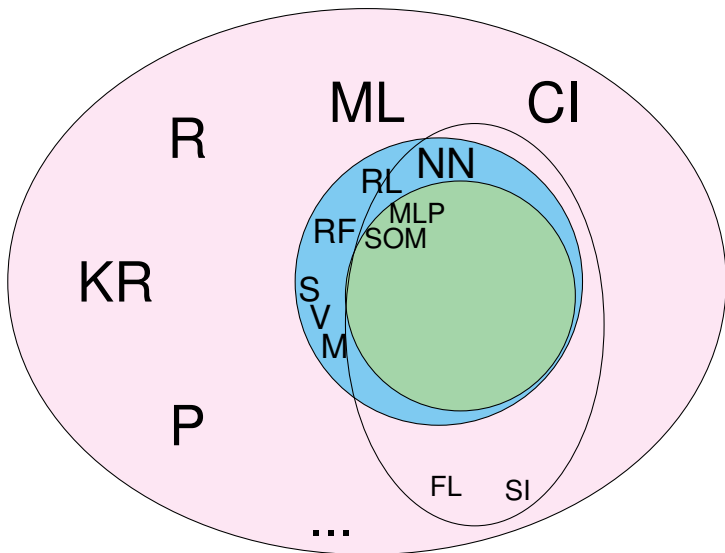


Inteligencia Artificial

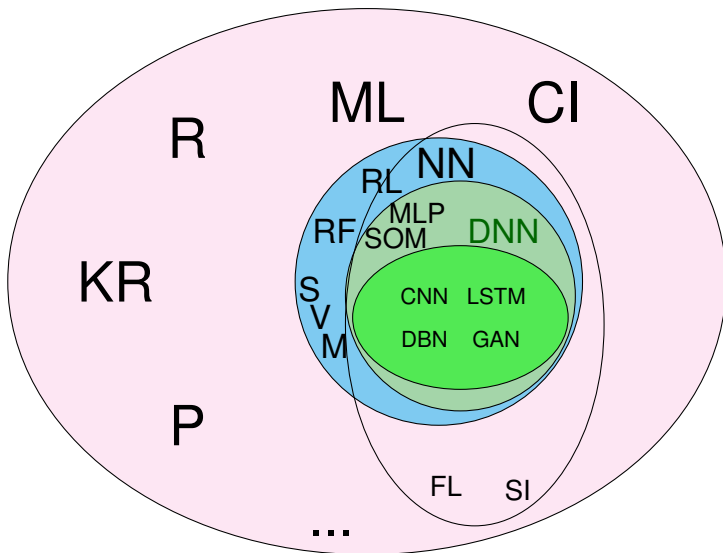




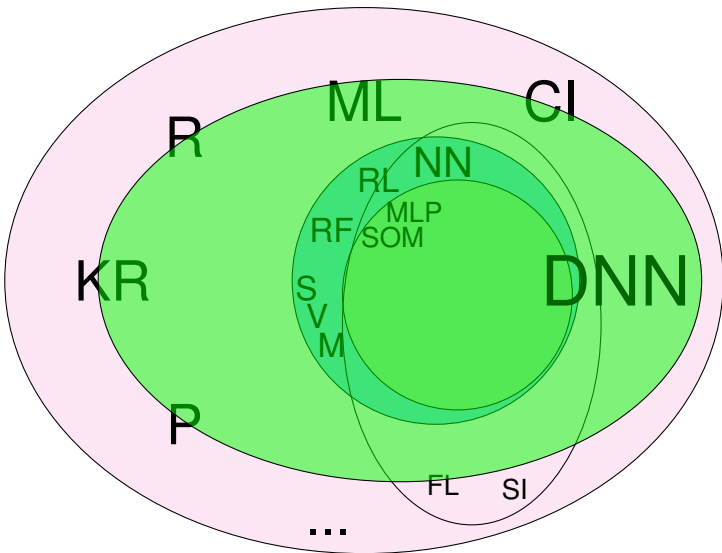
Inteligencia Artificial



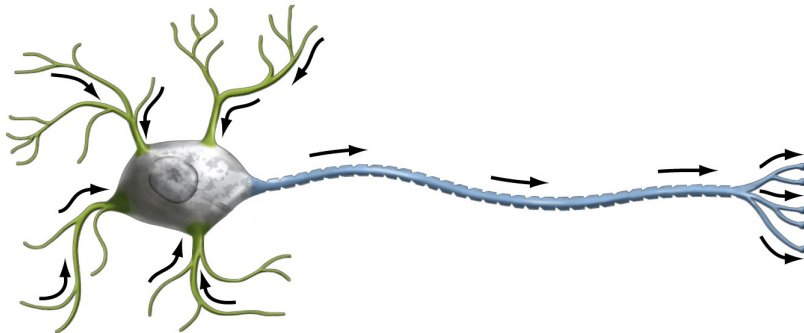
Inteligencia Artificial



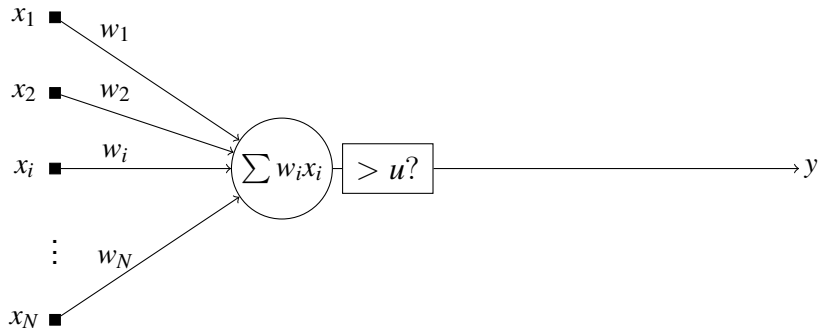
Inteligencia Artificial



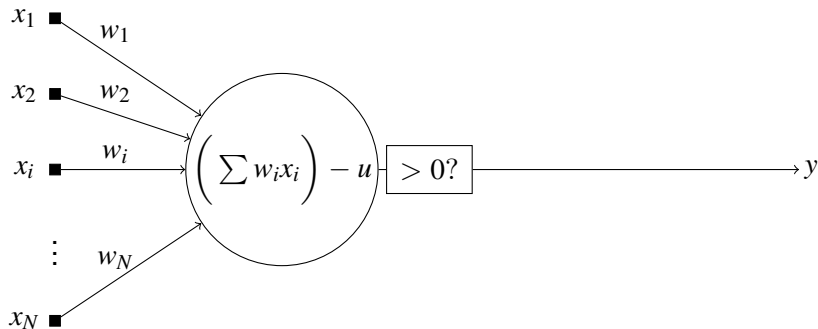
Neurona biológica



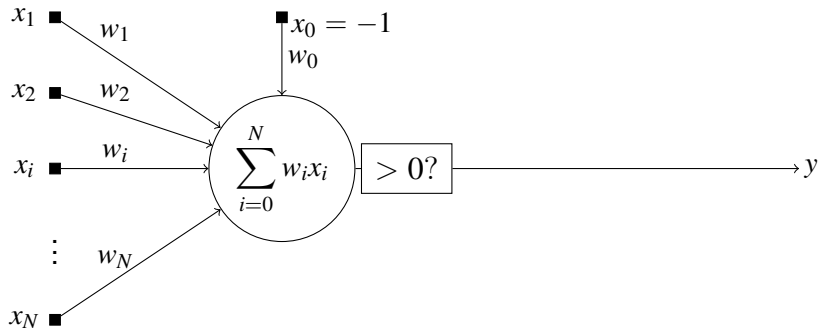
Modelo de neurona



Modelo de neurona



Modelo de neurona



Perceptrón simple: modelo simplificado

Producto interno con entrada extendida:

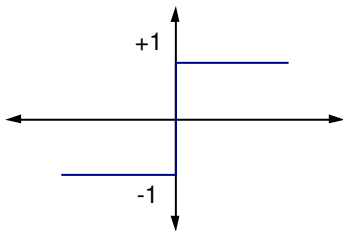
$$y = \phi \left(\sum_{i=0}^N w_i x_i \right) = \phi (\langle \mathbf{w}, \mathbf{x} \rangle)$$

Perceptrón simple: modelo simplificado

Producto interno con entrada extendida:

$$y = \phi \left(\sum_{i=0}^N w_i x_i \right) = \phi (\langle \mathbf{w}, \mathbf{x} \rangle)$$

Funciones de activación $\phi(z)$



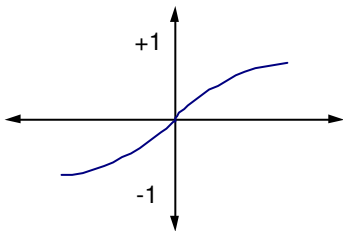
$$\text{sgn}(z) = \begin{cases} -1 & \text{si } z < 0 \\ +1 & \text{si } z \geq 0 \end{cases}$$

Perceptrón simple: modelo simplificado

Producto interno con entrada extendida:

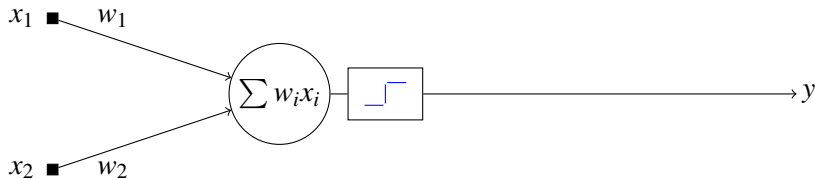
$$y = \phi \left(\sum_{i=0}^N w_i x_i \right) = \phi (\langle \mathbf{w}, \mathbf{x} \rangle)$$

Funciones de activación $\phi(z)$

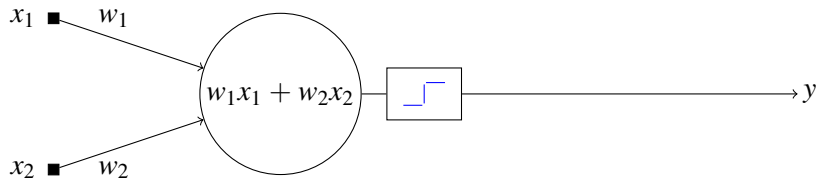


$$\text{sig}(z) = \frac{1 - e^{-az}}{1 + e^{-az}}$$

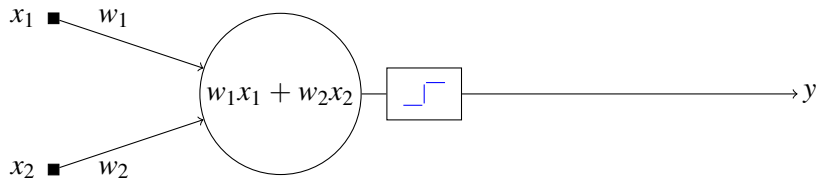
Perceptrón simple: ejemplo con 2 entradas



Perceptrón simple: ejemplo con 2 entradas

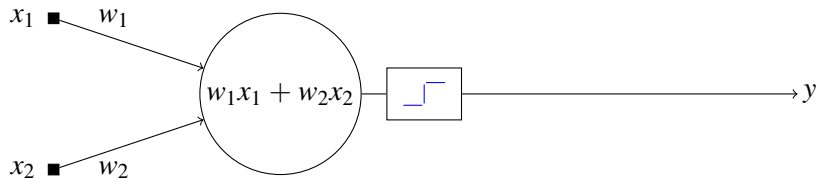


Perceptrón simple: ejemplo con 2 entradas



$$y = \text{sgn}(w_1x_1 + w_2x_2)$$

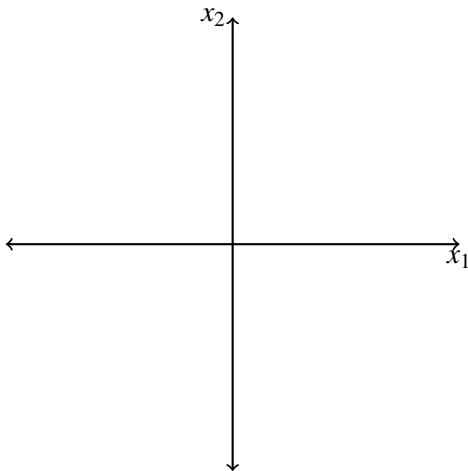
Perceptrón simple: ejemplo con 2 entradas



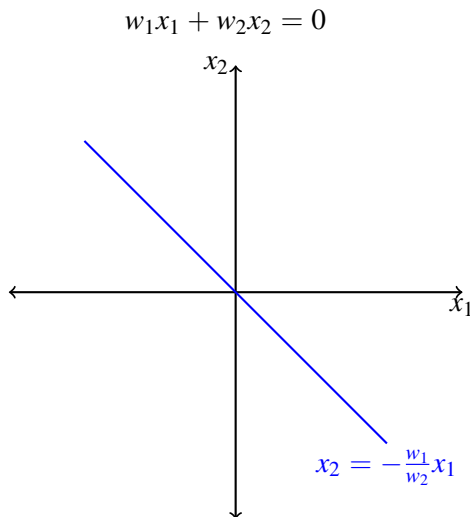
$$y = \text{sgn}(w_1x_1 + w_2x_2)$$

$$w_1x_1 + w_2x_2 > 0?$$

Perceptrón simple: ejemplo con 2 entradas

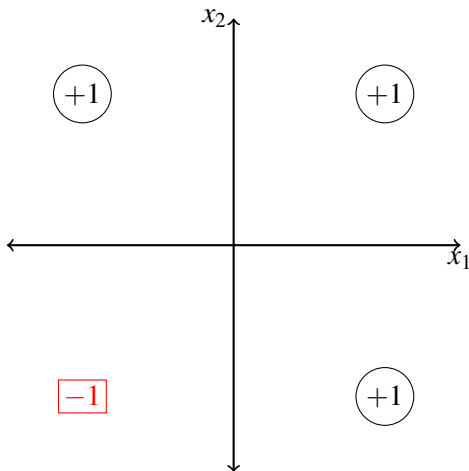


Perceptrón simple: ejemplo con 2 entradas



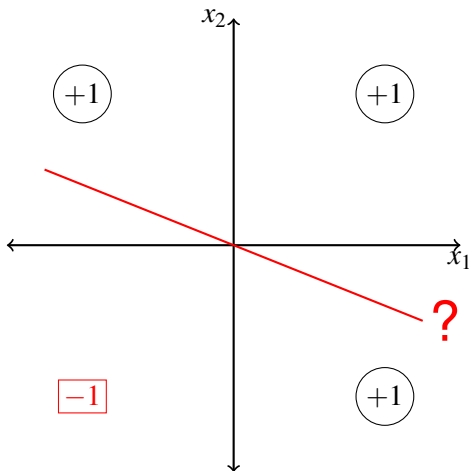
Perceptrón simple: ejemplo con 2 entradas

Caso del OR

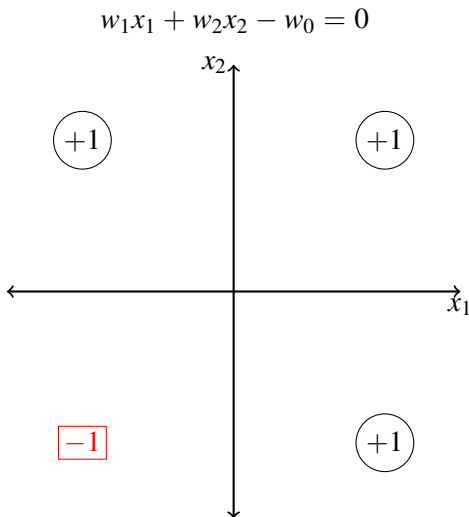


Perceptrón simple: ejemplo con 2 entradas

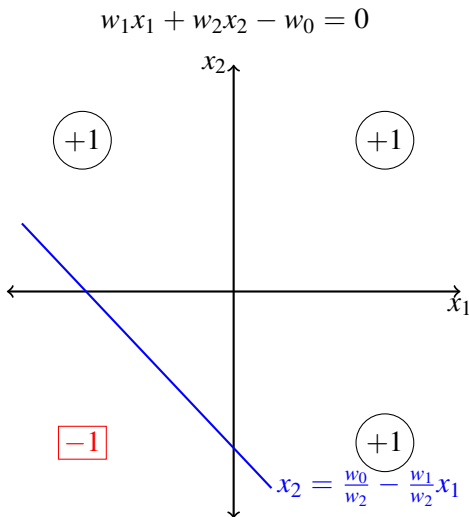
Caso del OR



Perceptrón simple: necesidad del bias



Perceptrón simple: necesidad del bias



Perceptrón simple: aprendizaje

¿Cómo aprende *solo* a partir de ejemplos?

Perceptrón simple: aprendizaje

Entrenamiento por corrección de error:

1. Inicialización al azar
2. Se muestran muchos ejemplos con las salidas esperadas y en cada caso:

Perceptrón simple: aprendizaje

Entrenamiento por corrección de error:

1. Inicialización al azar
2. Se muestran muchos ejemplos con las salidas esperadas y en cada caso:
 - Se obtiene la salida

Perceptrón simple: aprendizaje

Entrenamiento por corrección de error:

1. Inicialización al azar
2. Se muestran muchos ejemplos con las salidas esperadas y en cada caso:
 - Se obtiene la salida
 - **SI** la salida de la red es correcta
→ *no se hacen cambios*
(principio de mínima perturbación)

Perceptrón simple: aprendizaje

Entrenamiento por corrección de error:

1. Inicialización al azar
2. Se muestran muchos ejemplos con las salidas esperadas y en cada caso:
 - Se obtiene la salida
 - **SI** la salida de la red es correcta
→ *no se hacen cambios*
(principio de mínima perturbación)
 - **SI** la salida de la red es incorrecta
→ *penalización*
se actualizan los pesos en el sentido opuesto
al que contribuyó a la salida incorrecta

Perceptrón simple: aprendizaje

Entrenamiento por corrección de error:

1. Inicialización al azar
2. Se muestran muchos ejemplos con las salidas esperadas y en cada caso:
 - Se obtiene la salida

Perceptrón simple: aprendizaje

Entrenamiento por corrección de error:

1. Inicialización al azar $\rightarrow \mathbf{w}(1) \in [-0,5; 0,5]$
2. Se muestran muchos ejemplos con las salidas esperadas $\mathbf{x}(n)|y_d(n)$ y en cada caso:
 - Se obtiene la salida $\rightarrow y(n) = \phi(\langle \mathbf{w}(n), \mathbf{x}(n) \rangle)$

Perceptrón simple: aprendizaje

Entrenamiento por corrección de error:

1. Inicialización al azar $\rightarrow \mathbf{w}(1) \in [-0,5; 0,5]$
2. Se muestran muchos ejemplos con las salidas esperadas $\mathbf{x}(n)|y_d(n)$ y en cada caso:
 - Se obtiene la salida $\rightarrow y(n) = \phi(\langle \mathbf{w}(n), \mathbf{x}(n) \rangle)$
 - SI la salida de la red es correcta

Perceptrón simple: aprendizaje

Entrenamiento por corrección de error:

1. Inicialización al azar $\rightarrow \mathbf{w}(1) \in [-0,5; 0,5]$
2. Se muestran muchos ejemplos con las salidas esperadas $\mathbf{x}(n)|y_d(n)$ y en cada caso:
 - Se obtiene la salida $\rightarrow y(n) = \phi(\langle \mathbf{w}(n), \mathbf{x}(n) \rangle)$
 - **SI** $y(n) = y_d(n)$

Perceptrón simple: aprendizaje

Entrenamiento por corrección de error:

1. Inicialización al azar $\rightarrow \mathbf{w}(1) \in [-0,5; 0,5]$
2. Se muestran muchos ejemplos con las salidas esperadas $\mathbf{x}(n)|y_d(n)$ y en cada caso:
 - Se obtiene la salida $\rightarrow y(n) = \phi(\langle \mathbf{w}(n), \mathbf{x}(n) \rangle)$
 - **SI** $y(n) \neq y_d(n) \rightarrow \mathbf{w}(n+1) = \mathbf{w}(n)$

Perceptrón simple: aprendizaje

Entrenamiento por corrección de error:

1. Inicialización al azar $\rightarrow \mathbf{w}(1) \in [-0,5; 0,5]$
2. Se muestran muchos ejemplos con las salidas esperadas $\mathbf{x}(n)|y_d(n)$ y en cada caso:
 - Se obtiene la salida $\rightarrow y(n) = \phi(\langle \mathbf{w}(n), \mathbf{x}(n) \rangle)$
 - **SI** $y(n) = y_d(n) \rightarrow \mathbf{w}(n+1) = \mathbf{w}(n)$
 - **SI** la salida de la red es incorrecta

Perceptrón simple: aprendizaje

Entrenamiento por corrección de error:

1. Inicialización al azar $\rightarrow \mathbf{w}(1) \in [-0,5; 0,5]$
2. Se muestran muchos ejemplos con las salidas esperadas $\mathbf{x}(n)|y_d(n)$ y en cada caso:
 - Se obtiene la salida $\rightarrow y(n) = \phi(\langle \mathbf{w}(n), \mathbf{x}(n) \rangle)$
 - **SI** $y(n) = y_d(n) \rightarrow \mathbf{w}(n+1) = \mathbf{w}(n)$
 - **SI** $y(n) \neq y_d(n)$ (supongamos $x_i(n) > 0$)

Perceptrón simple: aprendizaje

Entrenamiento por corrección de error:

1. Inicialización al azar $\rightarrow \mathbf{w}(1) \in [-0,5; 0,5]$
2. Se muestran muchos ejemplos con las salidas esperadas $\mathbf{x}(n)|y_d(n)$ y en cada caso:
 - Se obtiene la salida $\rightarrow y(n) = \phi(\langle \mathbf{w}(n), \mathbf{x}(n) \rangle)$
 - **SI** $y(n) = y_d(n) \rightarrow \mathbf{w}(n+1) = \mathbf{w}(n)$
 - **SI** $y(n) \neq y_d(n)$
 SI $y_d(n) = -1$ pero $y(n) = +1$

Perceptrón simple: aprendizaje

Entrenamiento por corrección de error:

1. Inicialización al azar $\rightarrow \mathbf{w}(1) \in [-0,5; 0,5]$
2. Se muestran muchos ejemplos con las salidas esperadas $\mathbf{x}(n)|y_d(n)$ y en cada caso:
 - Se obtiene la salida $\rightarrow y(n) = \phi(\langle \mathbf{w}(n), \mathbf{x}(n) \rangle)$
 - **SI** $y(n) = y_d(n) \rightarrow \mathbf{w}(n+1) = \mathbf{w}(n)$
 - **SI** $y(n) \neq y_d(n)$
 - SI** $y_d(n) = -1$ pero $y(n) = +1 \rightarrow \mathbf{w}(n+1) = \mathbf{w}(n) - \eta \mathbf{x}(n)$
 - SI** $y_d(n) = +1$ pero $y(n) = -1 \rightarrow \mathbf{w}(n+1) = \mathbf{w}(n) + \eta \mathbf{x}(n)$

...podríamos simplificar usando $\epsilon(n) = \frac{y_d(n) - y(n)}{2}$

Perceptrón simple: aprendizaje

Entrenamiento por corrección de error:

1. Inicialización al azar $\rightarrow \mathbf{w}(1) \in [-0,5; 0,5]$
2. Se muestran muchos ejemplos con las salidas esperadas $\mathbf{x}(n)|y_d(n)$ y en cada caso:
 - Se obtiene la salida $\rightarrow y(n) = \phi(\langle \mathbf{w}(n), \mathbf{x}(n) \rangle)$
 - Se ajustan los pesos $\rightarrow \boxed{\mathbf{w}(n+1) = \mathbf{w}(n) + \eta \epsilon(n) \mathbf{x}(n)}$
3. Volver a 2 hasta satisfacer cierta tolerancia de error.

Perceptrón simple: método de gradiente (caso lineal)

Criterio del error instantáneo:

$$\xi(n) = \frac{1}{2} [y_d(n) - y(n)]^2 = \frac{1}{2} [y_d(n) - \langle \mathbf{w}(n), \mathbf{x}(n) \rangle]^2$$

Perceptrón simple: método de gradiente (caso lineal)

Criterio del error instantáneo:

$$\xi(n) = \frac{1}{2} [y_d(n) - y(n)]^2 = \frac{1}{2} [y_d(n) - \langle \mathbf{w}(n), \mathbf{x}(n) \rangle]^2$$

$$\nabla_{\mathbf{w}} \xi(n) = [y_d(n) - \langle \mathbf{w}(n), \mathbf{x}(n) \rangle] (-\mathbf{x}(n))$$

$$\nabla_{\mathbf{w}} \xi(n) = -e(n) \mathbf{x}(n)$$

Perceptrón simple: método de gradiente (caso lineal)

Criterio del error instantáneo:

$$\xi(n) = \frac{1}{2} [y_d(n) - y(n)]^2 = \frac{1}{2} [y_d(n) - \langle \mathbf{w}(n), \mathbf{x}(n) \rangle]^2$$

$$\nabla_{\mathbf{w}} \xi(n) = [y_d(n) - \langle \mathbf{w}(n), \mathbf{x}(n) \rangle] (-\mathbf{x}(n))$$

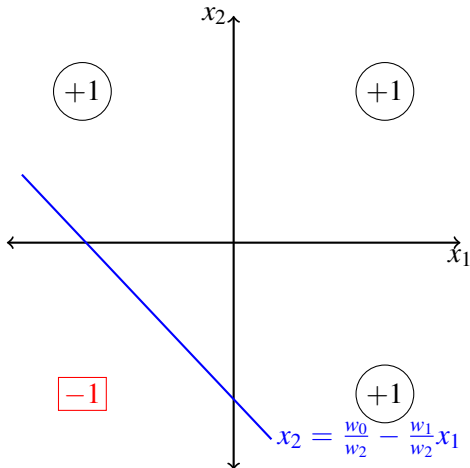
$$\nabla_{\mathbf{w}} \xi(n) = -e(n) \mathbf{x}(n)$$

reemplazando en: $\mathbf{w}(n+1) = \mathbf{w}(n) - \mu \nabla_{\mathbf{w}} \xi(n)$

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu e(n) \mathbf{x}(n)$$

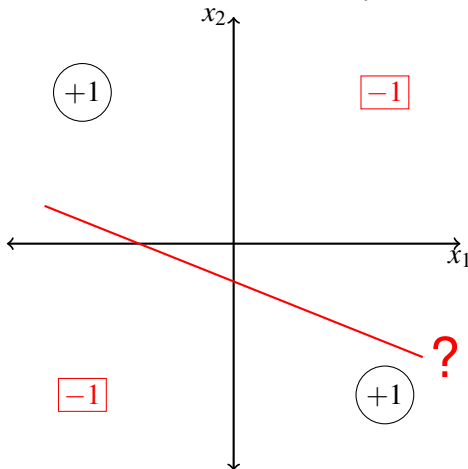
Espacio de soluciones del perceptrón simple

¿Qué tipo de problemas podemos resolver con una neurona?

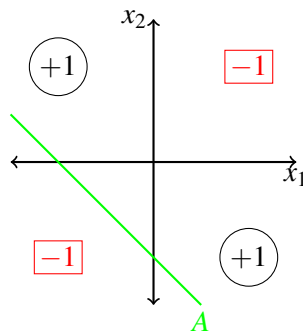


Espacio de soluciones del perceptrón simple

¿Podemos resolver este otro problema?



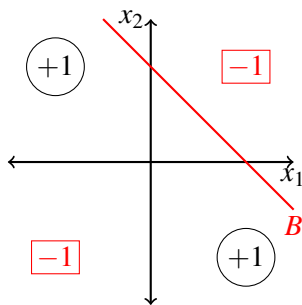
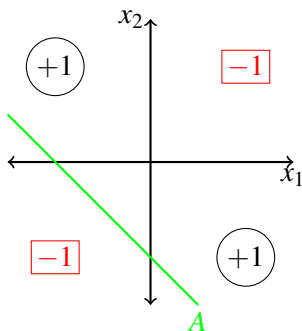
Combinación de perceptrones simples: A



Perceptrón A: $x_2 = -1 - x_1 = \frac{w_{A0}}{w_{A2}} - \frac{w_{A1}}{w_{A2}}x_1$

$$\rightarrow \left\{ \begin{array}{l} w_{A0} = -1 \\ w_{A1} = +1 \\ w_{A2} = +1 \end{array} \right\} \rightarrow y_A = \text{sgn}(x_2 + x_1 + 1)$$

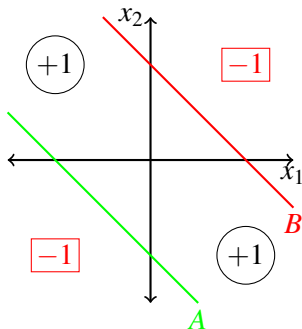
Combinación de perceptrones simples: B



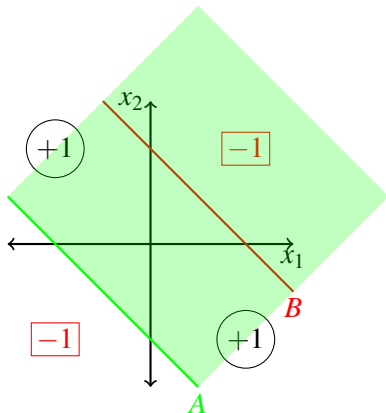
Perceptrón B: $x_2 = +1 - x_1$

$$\rightarrow \left\{ \begin{array}{l} w_{B0} = +1 \\ w_{B1} = +1 \\ w_{B2} = +1 \end{array} \right\} \rightarrow y_B = \text{sgn}(x_2 + x_1 - 1)$$

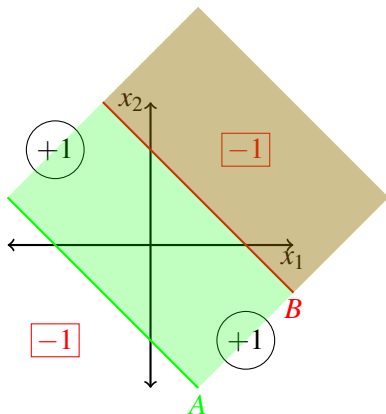
Combinación de perceptrones simples: C



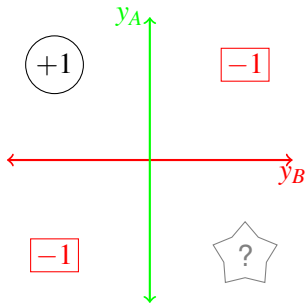
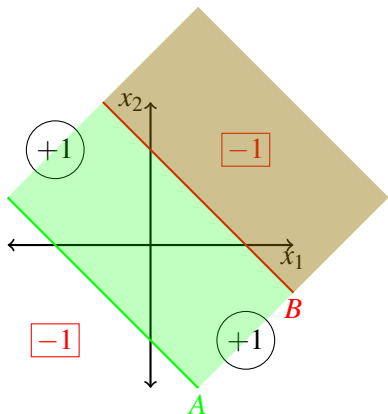
Combinación de perceptrones simples: C



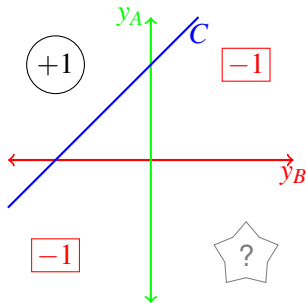
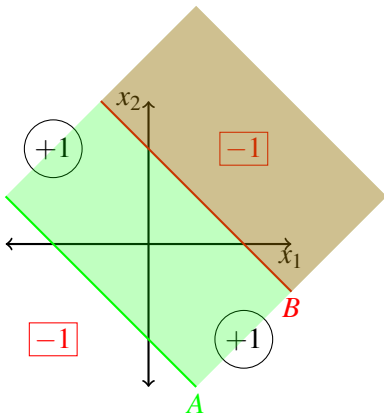
Combinación de perceptrones simples: C



Combinación de perceptrones simples: C



Combinación de perceptrones simples: C

Perceptrón C: $y_A = +1 + y_B$

$$\rightarrow \left\{ \begin{array}{l} w_{C0} = +1 \\ w_{C1} = -1 \\ w_{C2} = +1 \end{array} \right\} \rightarrow y_C = \text{sgn}(y_A - y_B - 1)$$

¿Cómo es la arquitectura de esta red neuronal?

$$\left. \begin{aligned} y_A &= \text{sgn}(x_2 + x_1 + 1) \\ y_B &= \text{sgn}(x_2 + x_1 - 1) \end{aligned} \right\} \rightarrow y_C = \text{sgn}(y_A - y_B - 1)$$

$x_1 \longrightarrow$ 

A

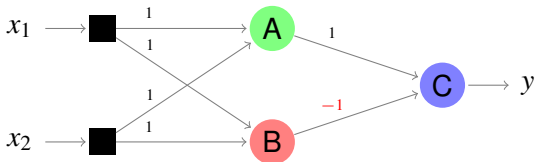
$x_2 \longrightarrow$ 

B

C $\longrightarrow y$

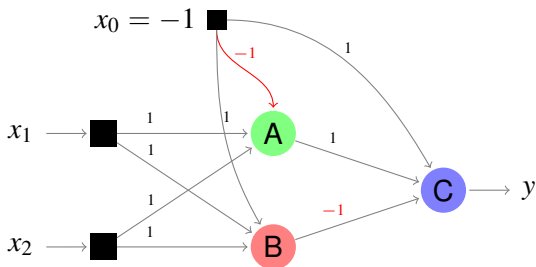
¿Cómo es la arquitectura de esta red neuronal?

$$\left. \begin{aligned} y_A &= \text{sgn}(x_2 + x_1 + 1) \\ y_B &= \text{sgn}(x_2 + x_1 - 1) \end{aligned} \right\} \rightarrow y_C = \text{sgn}(y_A - y_B - 1)$$

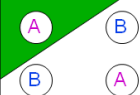
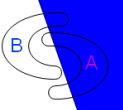
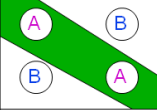
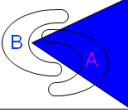
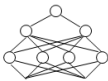
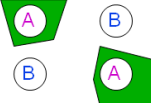
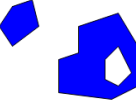


¿Cómo es la arquitectura de esta red neuronal?

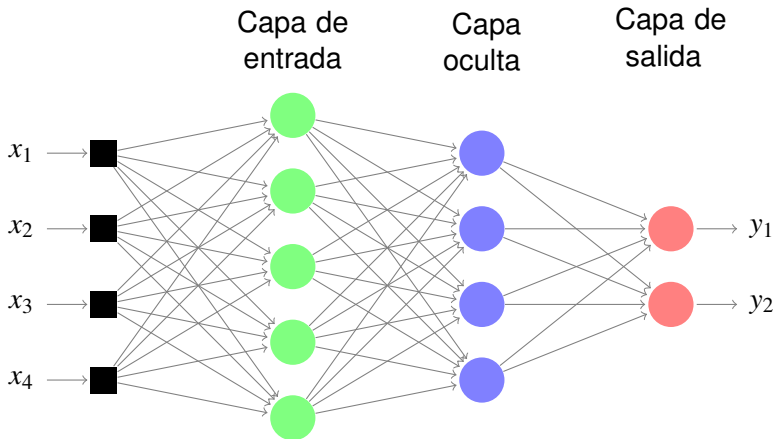
$$\left. \begin{aligned} y_A &= \text{sgn}(x_2 + x_1 + 1) \\ y_B &= \text{sgn}(x_2 + x_1 - 1) \end{aligned} \right\} \rightarrow y_C = \text{sgn}(y_A - y_B - 1)$$



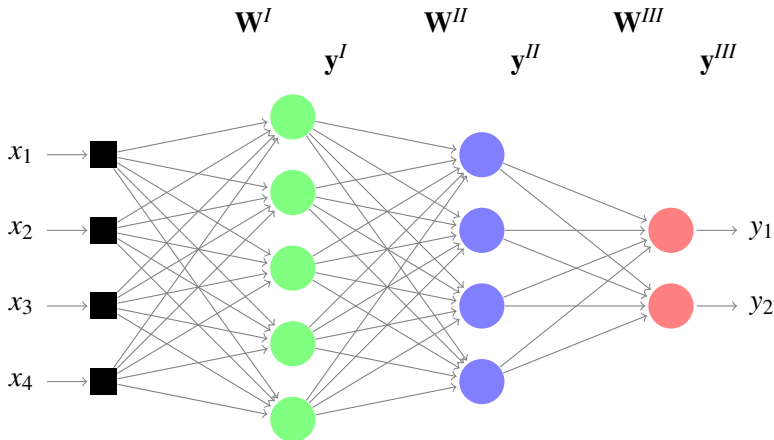
Regiones de decisión

Estructura	Tipos de regiones de decisión	Problema XOR	Separación en clases	Formas regiones más generales
Una capa 	hemiplano limitado por hiperplano			
Dos capas 	Regiones convexas abiertas o cerradas			
Tres capas 	Arbitrarias (Complejidad limitada por N°. de Nodos)			

Arquitectura del perceptrón multicapa



Arquitectura del perceptrón multicapa



Cálculo de las salidas en cada capa

- Capa I:

$$v_j^I = \langle \mathbf{w}_j^I, \mathbf{x} \rangle = \sum_{i=0}^N w_{ji}^I x_i \quad (\text{completo } \mathbf{v}^I = \mathbf{W}^I \mathbf{x})$$

Cálculo de las salidas en cada capa

- Capa I:

$$v_j^I = \langle \mathbf{w}_j^I, \mathbf{x} \rangle = \sum_{i=0}^N w_{ji}^I x_i \quad (\text{completo } \mathbf{v}^I = \mathbf{W}^I \mathbf{x})$$

$$y_j^I = \phi(v_j^I) = \frac{2}{1 + e^{-bv_j^I}} - 1 \quad (\text{simétrica } \pm 1)$$

Cálculo de las salidas en cada capa

- Capa I:

$$v_j^I = \langle \mathbf{w}_j^I, \mathbf{x} \rangle = \sum_{i=0}^N w_{ji}^I x_i \quad (\text{completo } \mathbf{v}^I = \mathbf{W}^I \mathbf{x})$$

$$y_j^I = \phi(v_j^I) = \frac{2}{1 + e^{-bv_j^I}} - 1 \quad (\text{simétrica } \pm 1)$$

- Capa II:

$$v_j^{II} = \langle \mathbf{w}_j^{II}, \mathbf{y}^I \rangle \rightarrow y_j^{II} = \phi(v_j^{II})$$

- Capa III:

$$v_j^{III} = \langle \mathbf{w}_j^{III}, \mathbf{y}^{II} \rangle \rightarrow y_j^{III} = \phi(v_j^{III}) = y_j$$

Aplicación del gradiente (caso general)

$$\xi(n) = \frac{1}{2} \sum_{j=1}^M e_j^2(n)$$

Aplicación del gradiente (caso general)

$$\Delta w_{ji}(n) = -\mu \frac{\partial \xi(n)}{\partial w_{ji}(n)}$$

Aplicación del gradiente (caso general)

$$\Delta w_{ji}(n) = -\mu \frac{\partial \xi(n)}{\partial w_{ji}(n)}$$

$$\frac{\partial \xi(n)}{\partial w_{ji}(n)} = \frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \frac{\partial v_j(n)}{\partial w_{ji}(n)}$$

Aplicación del gradiente (caso general)

$$\Delta w_{ji}(n) = -\mu \frac{\partial \xi(n)}{\partial w_{ji}(n)}$$

$$\frac{\partial \xi(n)}{\partial w_{ji}(n)} = \frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \boxed{\frac{\partial v_j(n)}{\partial w_{ji}(n)}}$$

$$\frac{\partial v_j(n)}{\partial w_{ji}(n)} = \frac{\partial \sum_{i=0}^N w_{ji}(n) y_i(n)}{\partial w_{ji}(n)} = y_i(n)$$

Retropropagación en la capa III (salida)

$$\Delta w_{ji}^{III}(n) = \mu \delta_j^{III}(n) y_i^{II}(n)$$

Retropropagación en la capa III (salida)

$$\Delta w_{ji}^{III}(n) = \mu \delta_j^{III}(n) y_i^{II}(n)$$

$$\delta_j^{III}(n) = -\frac{\partial \xi(n)}{\partial y_j^{III}(n)} \frac{1}{2} (1 + y_j^{III}(n)) (1 - y_j^{III}(n))$$

$$\delta_j^{III}(n) = -\frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j^{III}(n)} \frac{1}{2} (1 + y_j^{III}(n)) (1 - y_j^{III}(n))$$

Retropropagación en la capa III (salida)

$$\delta_j^{III}(n) = -\frac{\partial \left\{ \frac{1}{2} \sum_j e_j^2(n) \right\}}{\partial e_j(n)} \cdot \frac{\partial \left\{ d_j^{III}(n) - y_j^{III}(n) \right\}}{\partial y_j^{III}(n)} \cdot \frac{1}{2}(1 + y_j^{III}(n))(1 - y_j^{III}(n))$$

$$\delta_j^{III}(n) = \frac{1}{2}e_j(n)(1 + y_j^{III}(n))(1 - y_j^{III}(n))\star$$

$$\Delta w_{ji}^{III}(n) = \eta e_j(n)(1 + y_j^{III}(n))(1 - y_j^{III}(n))y_i^{II}(n)$$

Retropropagación en la capa II (oculta)

$$\Delta w_{ji}^H(n) = \mu \delta_j^H(n) y_i^I(n)$$

Retropropagación en la capa II (oculta)

$$\Delta w_{ji}^H(n) = \mu \delta_j^H(n) y_i^I(n)$$

$$\delta_j^H(n) = -\frac{\partial \xi(n)}{\partial y_j^H(n)} \frac{1}{2} (1 + y_j^H(n))(1 - y_j^H(n))$$

$$\delta_j^H(n) = -\frac{\partial \left\{ \frac{1}{2} \sum_k e_k^2(n) \right\}}{\partial y_j^H(n)} \frac{1}{2} (1 + y_j^H(n))(1 - y_j^H(n))$$

$$\delta_j^H(n) = -\frac{1}{2} \sum_k \frac{\partial e_k^2(n)}{\partial y_j^H(n)} \frac{1}{2} (1 + y_j^H(n))(1 - y_j^H(n))$$

$$\delta_j^H(n) = -\sum_k e_k(n) \frac{\partial e_k(n)}{\partial y_j^H(n)} \frac{1}{2} (1 + y_j^H(n))(1 - y_j^H(n))$$

Retropropagación en la capa II (oculta)

$$\delta_j^H(n) = - \sum_k e_k(n) \frac{\partial e_k(n)}{\partial y_k^{III}(n)} \frac{\partial y_k^{III}(n)}{\partial v_k^{III}(n)} \frac{\partial v_k^{III}(n)}{\partial y_j^H(n)} \frac{1}{2} (1 + y_j^H(n)) (1 - y_j^H(n))$$

$$\delta_j^H(n) = - \sum_k e_k(n) \cdot \frac{\partial \{d_k^{III}(n) - y_k^{III}(n)\}}{\partial y_k^{III}(n)} \cdot \frac{1}{2} (1 + y_k^{III}(n)) (1 - y_k^{III}(n)) \cdot$$

$$\cdot \frac{\partial \{ \sum_j w_{kj}^{III} y_j^H(n) \}}{\partial y_j^H(n)} \cdot \frac{1}{2} (1 + y_j^H(n)) (1 - y_j^H(n))$$

$$\delta_j^H(n) = - \sum_k e_k(n) \cdot (-1) \cdot \frac{1}{2} (1 + y_k^{III}(n)) (1 - y_k^{III}(n)) \cdot$$

$$\cdot w_{kj}^{III} \cdot \frac{1}{2} (1 + y_j^H(n)) (1 - y_j^H(n))$$

Retropropagación en la capa II (oculta)

$$\delta_j^H(n) = \sum_k e_k(n) \cdot \frac{1}{2}(1 + y_k^{III}(n))(1 - y_k^{III}(n)) \cdot w_{kj}^{III} \cdot \frac{1}{2}(1 + y_j^H(n))(1 - y_j^H(n))$$

Pero de la capa III★ sabemos que

$$\delta_k^{III}(n) = \frac{1}{2}e_k(n)(1 + y_k^{III}(n))(1 - y_k^{III}(n))$$

Retropropagación en la capa II (oculta)

$$\delta_j^H(n) = \sum_k e_k(n) \cdot \frac{1}{2}(1 + y_k^{III}(n))(1 - y_k^{III}(n)) \cdot w_{kj}^{III} \cdot \frac{1}{2}(1 + y_j^H(n))(1 - y_j^H(n))$$

Pero de la capa III★ sabemos que

$$\delta_k^{III}(n) = \frac{1}{2}e_k(n)(1 + y_k^{III}(n))(1 - y_k^{III}(n))$$

Reemplazando

$$\delta_j^H(n) = \sum_k \delta_k^{III}(n)w_{kj}^{III} \cdot \frac{1}{2}(1 + y_j^H(n))(1 - y_j^H(n))$$

Retropropagación en la capa II (oculta)

Volviendo a

$$\Delta w_{ji}^I(n) = \mu \delta_j^I(n) y_i^I(n)$$

Tenemos

$$\Delta w_{ji}^I(n) = \eta \left[\sum_k \delta_k^I w_{kj}^I(n) \right] (1 + y_j^I(n))(1 - y_j^I(n)) y_i^I(n)$$

Retropropagación en la capa II (oculta)

Volviendo a

$$\Delta w_{ji}^H(n) = \mu \delta_j^H(n) y_i^I(n)$$

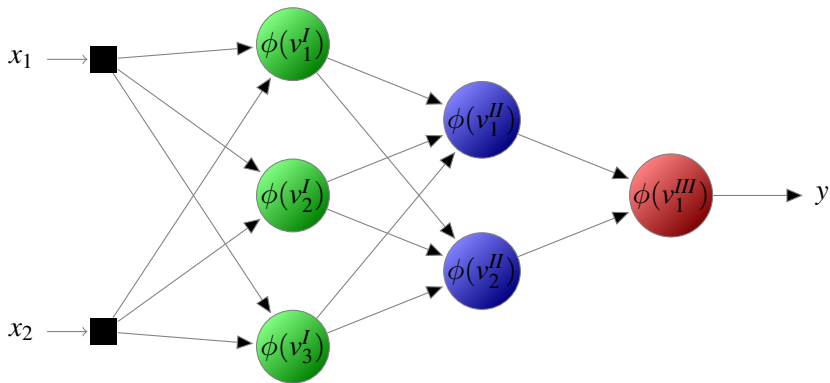
Tenemos

$$\Delta w_{ji}^H(n) = \eta \left[\sum_k \delta_k^H w_{kj}^H(n) \right] (1 + y_j^H(n))(1 - y_j^H(n)) y_i^I(n)$$

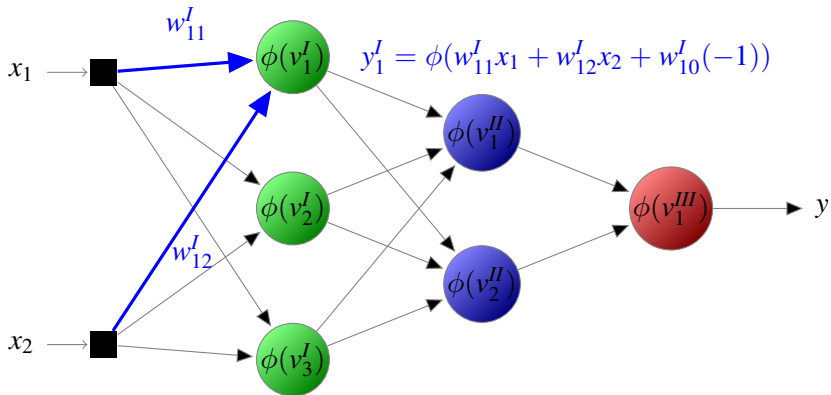
Generalización para la capa “p”

$$\Delta w_{ji}^{(p)} = \eta \left\langle \delta^{(p+1)}, \mathbf{w}_j^{(p+1)} \right\rangle (1 + y_j^{(p)}) (1 - y_j^{(p)}) y_i^{(p-1)}$$

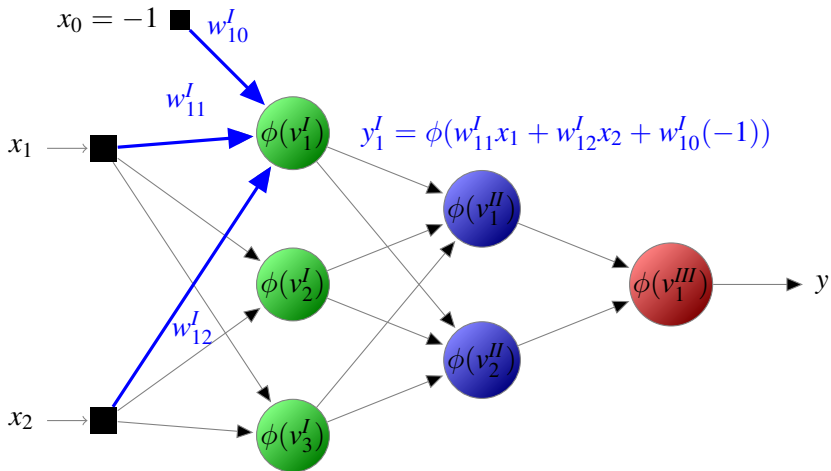
Resumen: propagación hacia adelante



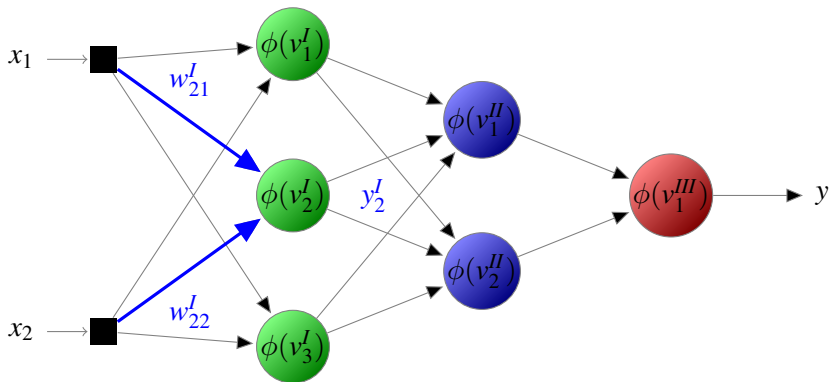
Resumen: propagación hacia adelante



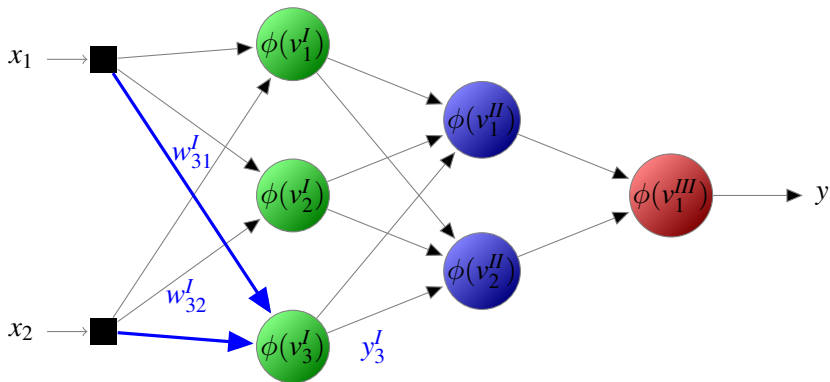
Resumen: propagación hacia adelante



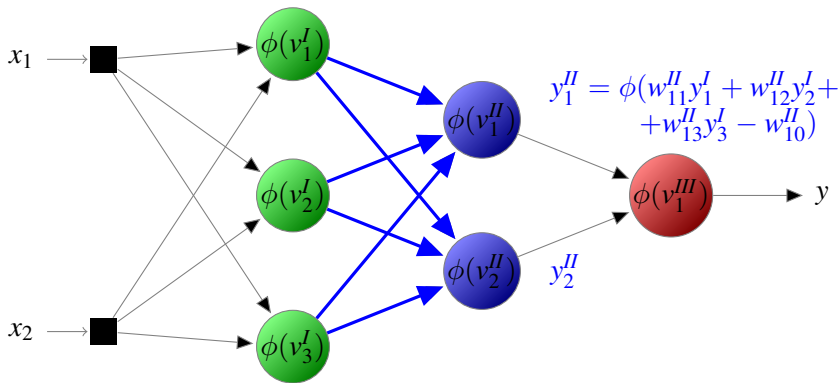
Resumen: propagación hacia adelante



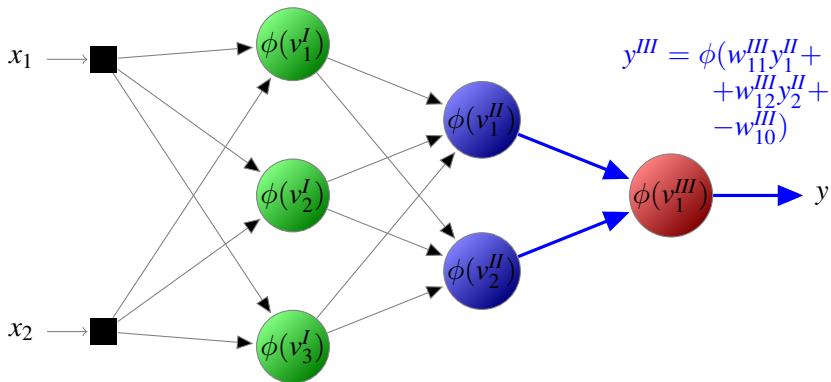
Resumen: propagación hacia adelante



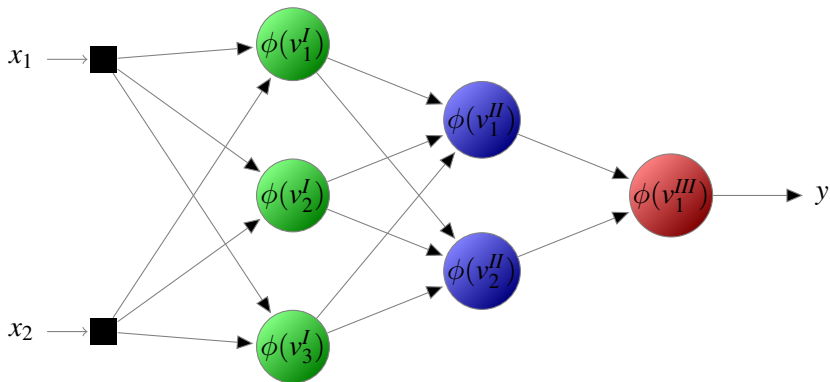
Resumen: propagación hacia adelante



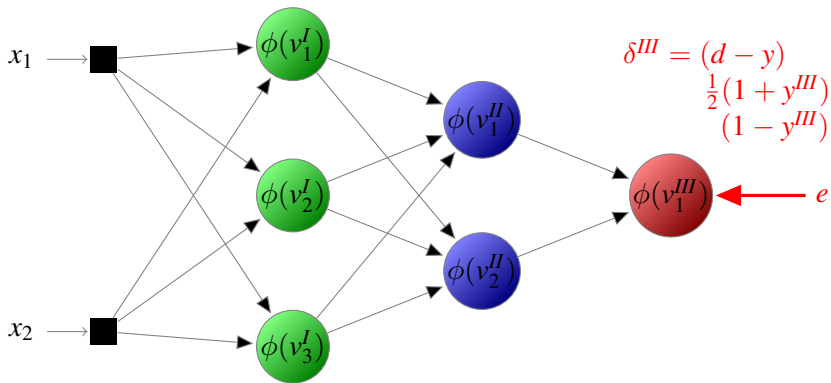
Resumen: propagación hacia adelante



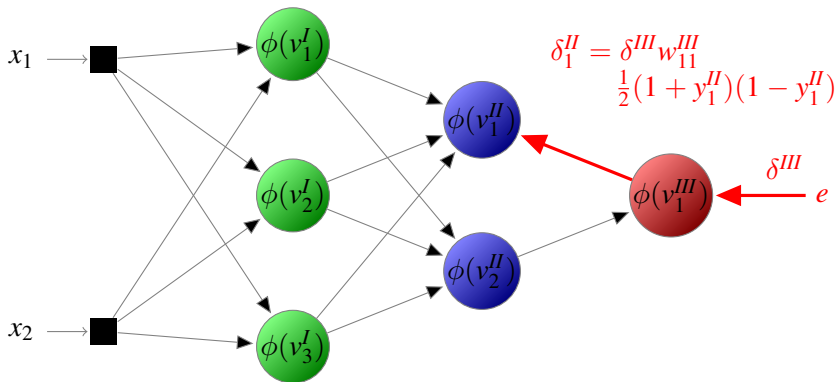
Resumen: propagación hacia **atras**



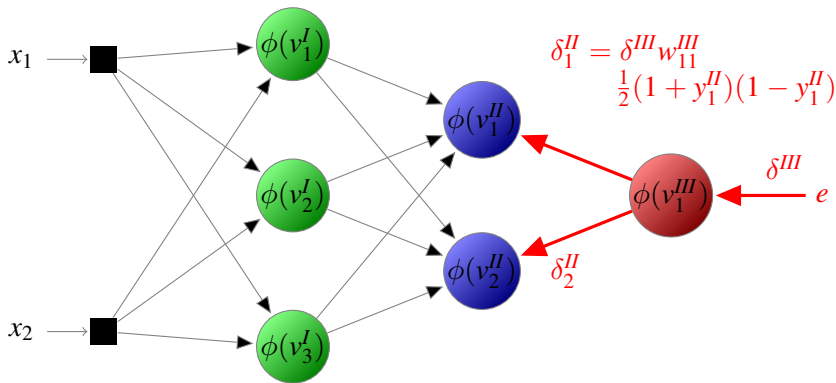
Resumen: propagación hacia **atras**



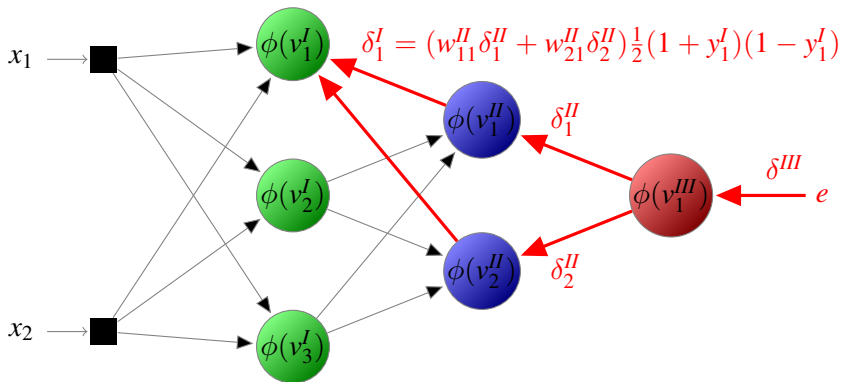
Resumen: propagación hacia **atras**



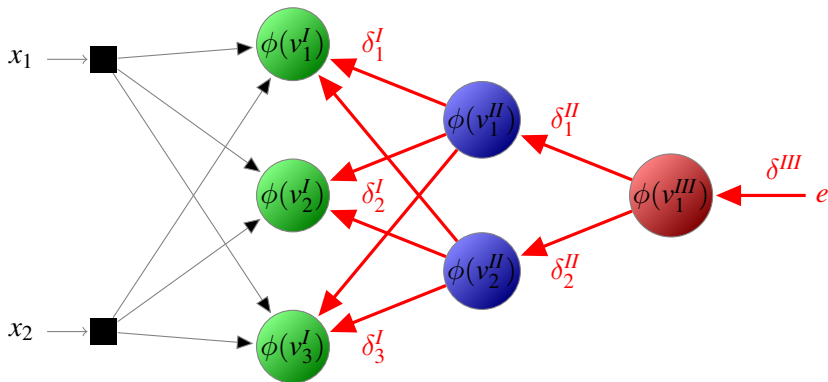
Resumen: propagación hacia **atras**



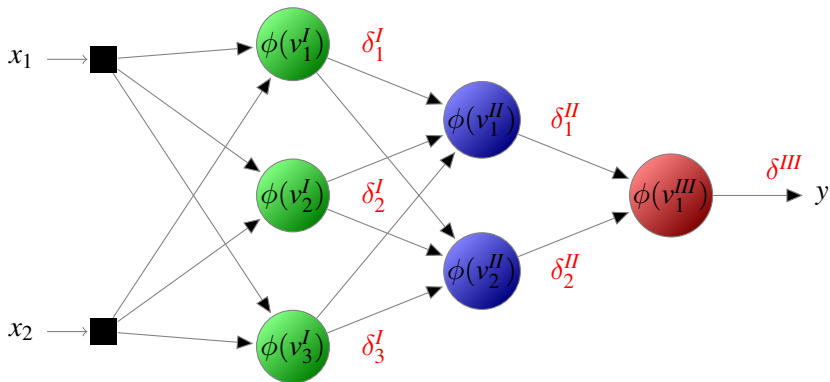
Resumen: propagación hacia **atras**



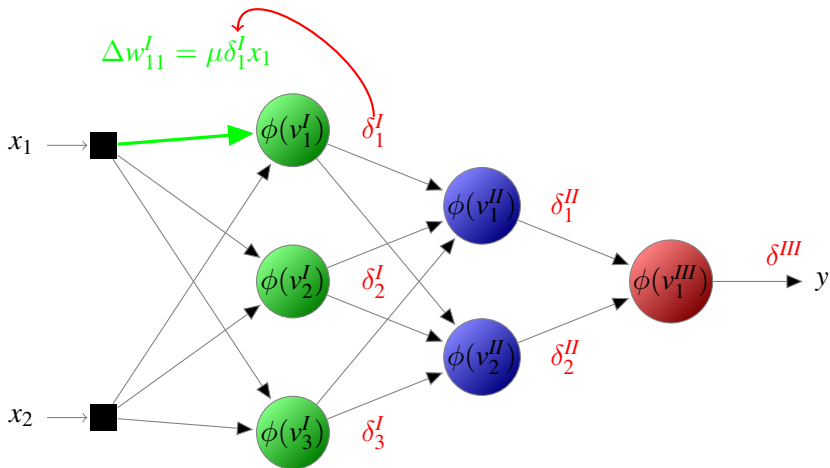
Resumen: propagación hacia **atras**



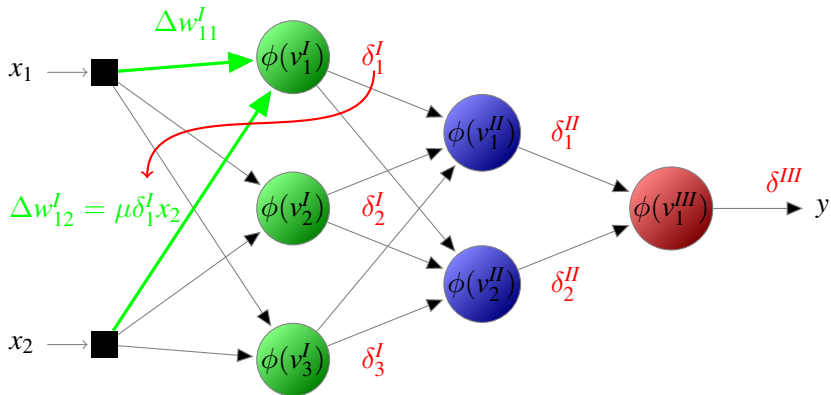
Resumen: ajuste de pesos



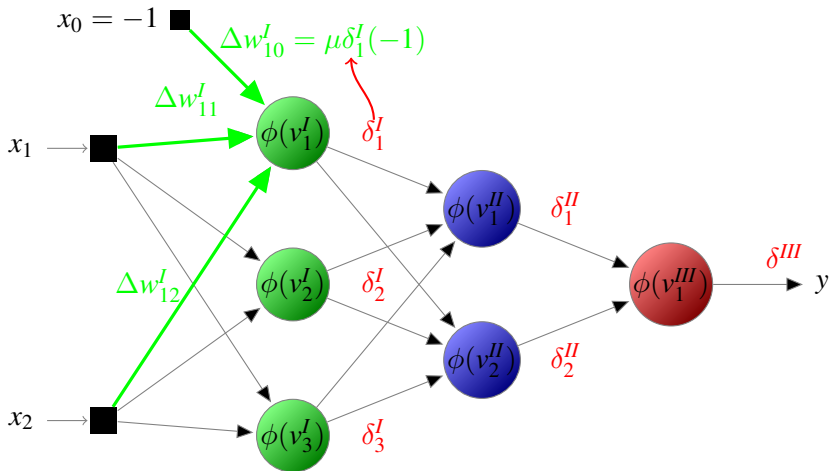
Resumen: ajuste de pesos



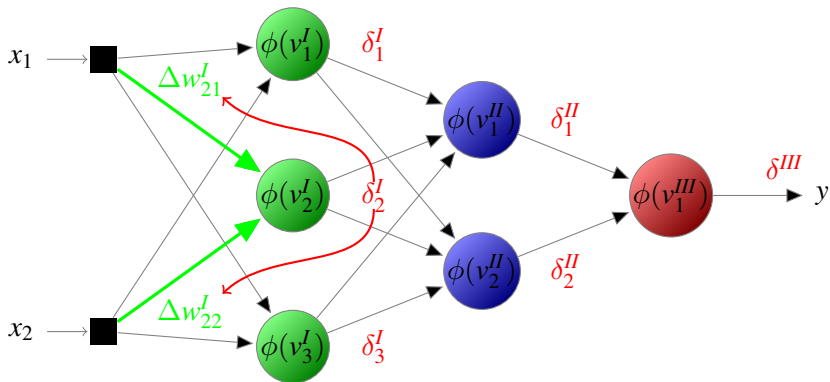
Resumen: ajuste de pesos



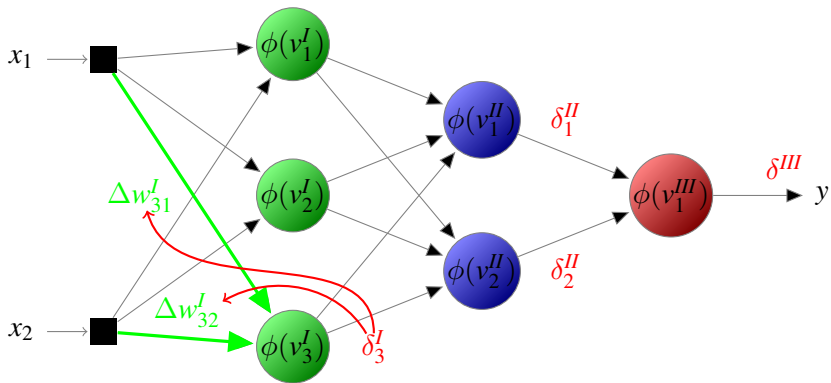
Resumen: ajuste de pesos



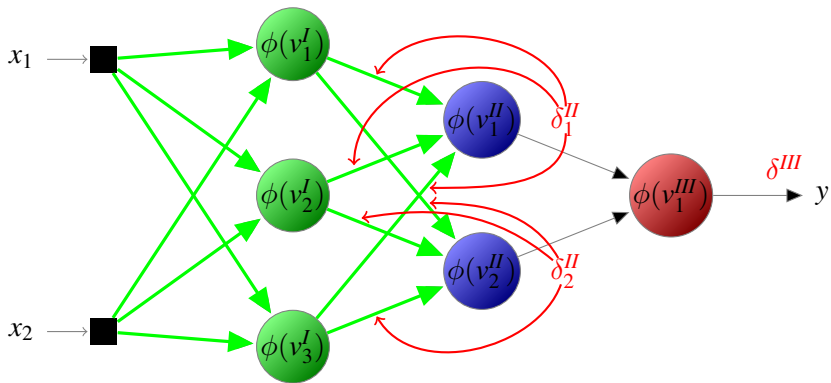
Resumen: ajuste de pesos



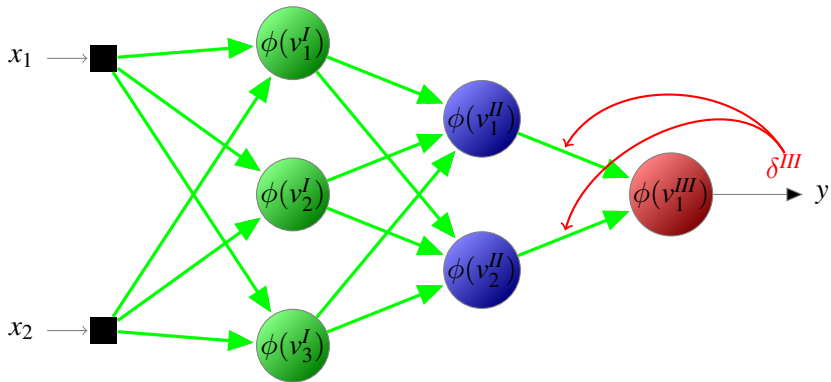
Resumen: ajuste de pesos



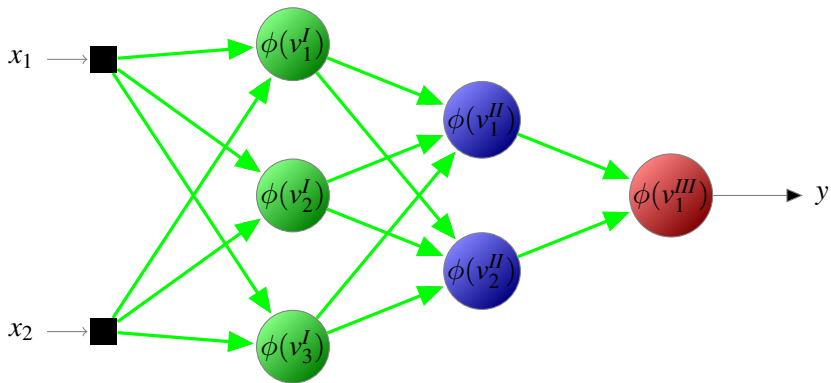
Resumen: ajuste de pesos



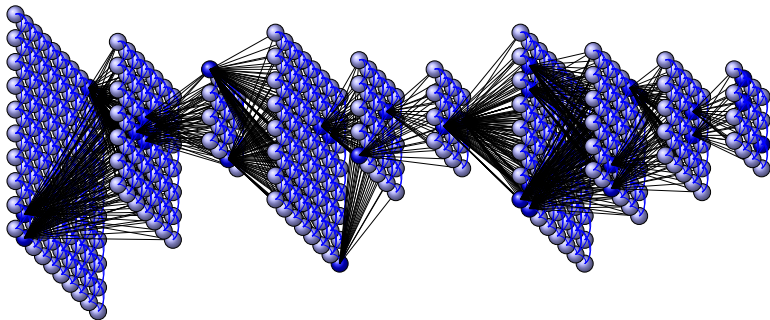
Resumen: ajuste de pesos



Resumen: ajuste de pesos



¿Qué pasa si conectamos muuuchas neuronas?



Qué pasa si conectamos muuuchas neuronas?

- Necesidad de contar con grandes cantidades de datos para entrenar

Qué pasa si conectamos muuuchas neuronas?

- Necesidad de contar con grandes cantidades de datos para entrenar
← Revolución de los **grandes datos**

Qué pasa si conectamos muuuchas neuronas?

- Necesidad de contar con grandes cantidades de datos para entrenar
← Revolución de los **grandes datos**
- Alto costo computacional

Qué pasa si conectamos muuuchas neuronas?

- Necesidad de contar con grandes cantidades de datos para entrenar
 - ← Revolución de los **grandes datos**
- Alto costo computacional
 - ← Unidades de procesamiento gráfico **GPUs**

Qué pasa si conectamos muuuchas neuronas?

- Necesidad de contar con grandes cantidades de datos para entrenar
 - ← Revolución de los **grandes datos**
- Alto costo computacional
 - ← Unidades de procesamiento gráfico **GPUs**
- Desvanecimiento del gradiente

Qué pasa si conectamos muuuchas neuronas?

- Necesidad de contar con grandes cantidades de datos para entrenar
 - ← Revolución de los **grandes datos**
- Alto costo computacional
 - ← Unidades de procesamiento gráfico **GPUs**
- Desvanecimiento del gradiente ←
 - Pre-entrenamiento no-supervisado,
 - Auto-codificadores con máquinas de Boltzmann,
 - Normalización por lotes,
 - Redes residuales,
 - Otros condimentos: **ReLU**, dropout, pooling...

Programación diferenciable

También conocida como:

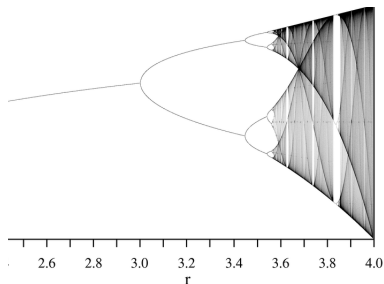
- ∂P (differentiable programming)
- Automatic Differentiation
- Algorithmic Differentiation
- Dynamic Computational Graphs
- *autodiff*
- Symbolic Differentiation

Programación diferenciable: mapa logístico

Supongamos que tenemos que calcular las derivadas del mapa logístico¹

$$\ell_{n+1} = r\ell_n(1 - \ell_n),$$

con respecto a x , a lo largo de las iteraciones n .



¹ A.G. Baydin et al. Automatic differentiation in machine learning: a survey. JMLR, 2017.

Programación diferenciable: mapa logístico

Con $r = 4$, por ejemplo, tenemos

$$\ell_{n+1} = 4\ell_n(1 - \ell_n), \ell_1 = x$$

$$1 \quad x$$

$$2 \quad 4x(1 - x)$$

$$3 \quad 16x(1-x)(1-2x)^2$$

$$4 \quad \frac{64x(1-x)(1-2x)^2}{(1 - 8x + 8x^2)^2}$$

Programación diferenciable: mapa logístico

Con $r = 4$, por ejemplo, tenemos

$$\ell_{n+1} = 4\ell_n(1 - \ell_n), \ell_1 = x$$

1	x	1
2	$4x(1-x)$	$4(1-x) - 4x$
3	$16x(1-x)(1-2x)^2$	$16(1-x)(1-2x)^2 - 16x(1-2x)^2 - 64x(1-x)(1-2x)$
4	$64x(1-x)(1-2x)^2(1-8x+8x^2)^2$	$128x(1-x)(-8+16x)(1-2x)^2(1-8x+8x^2) + 64(1-x)(1-2x)^2(1-8x+8x^2)^2 - 64x(1-2x)^2(1-8x+8x^2)^2 - 256x(1-x)(1-2x)(1-8x+8x^2)^2$

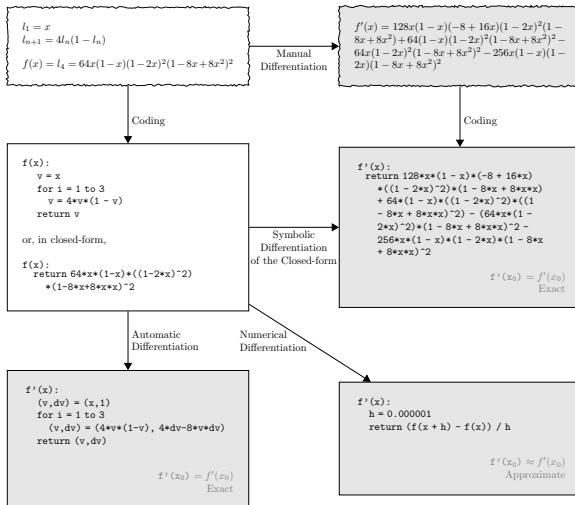
Programación diferenciable: mapa logístico

Con $r = 4$, por ejemplo, tenemos

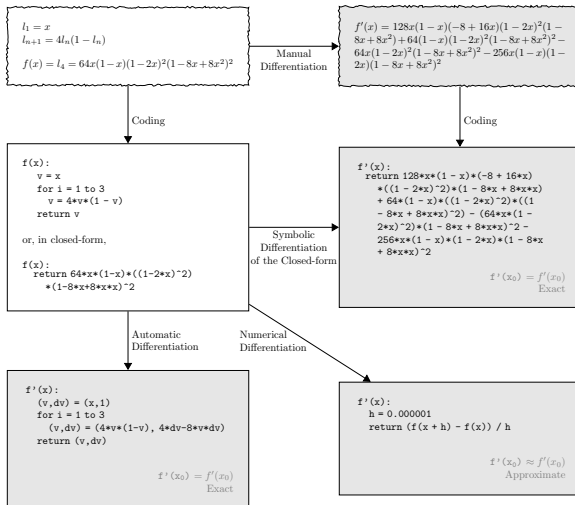
$$\ell_{n+1} = 4\ell_n(1 - \ell_n), \ell_1 = x$$

1	x	1	1
2	$4x(1-x)$	$4(1-x) - 4x$	$4 - 8x$
3	$16x(1-x)(1-2x)^2$	$16(1-x)(1-2x)^2 - 16x(1-2x)^2 - 64x(1-x)(1-2x)$	$16(1 - 10x + 24x^2 - 16x^3)$
4	$64x(1-x)(1-2x)^2(1-8x+8x^2)^2$	$128x(1-x)(-8+16x)(1-2x)^2(1-8x+8x^2) + 64(1-x)(1-2x)^2(1-8x+8x^2)^2 - 64x(1-2x)^2(1-8x+8x^2)^2 - 256x(1-x)(1-2x)(1-8x+8x^2)^2$	$64(1 - 42x + 504x^2 - 2640x^3 + 7040x^4 - 9984x^5 + 7168x^6 - 2048x^7)$

Derivación automática: que no es



Derivación automática: que no es



Derivación automática: que **no es**

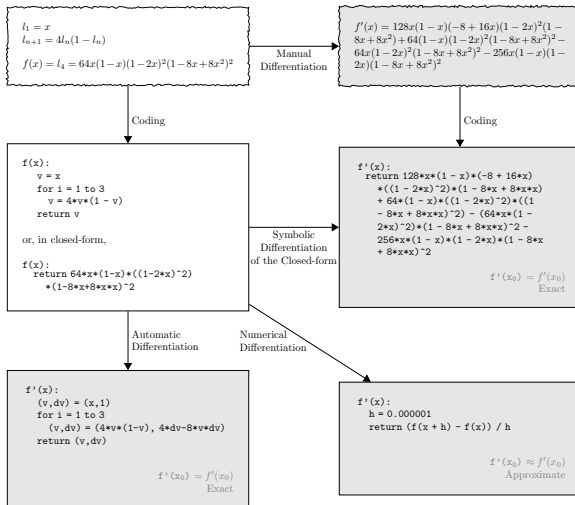
$$l_1 = x$$

$$l_{n+1} = 4l_n(1 - l_n)$$

$$f(x) = l_4 = 64x(1-x)(1-2x)^2(1-8x+8x^2)^2$$

Coding

Derivación automática: que no es



Derivación automática: que **no es**

Coding

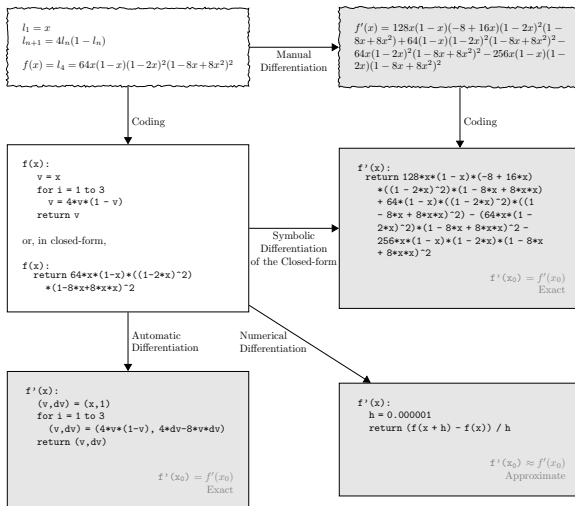
```
f(x):  
  v = x  
  for i = 1 to 3  
    v = 4*v*(1 - v)  
  return v
```

or, in closed-form,

```
f(x):  
  return 64*x*(1-x)*((1-2*x)^2)  
         *(1-8*x+8*x*x)^2
```

Symbolic
Differentiation
of the Closed-form

Derivación automática: que no es



Derivación automática: que **no es**

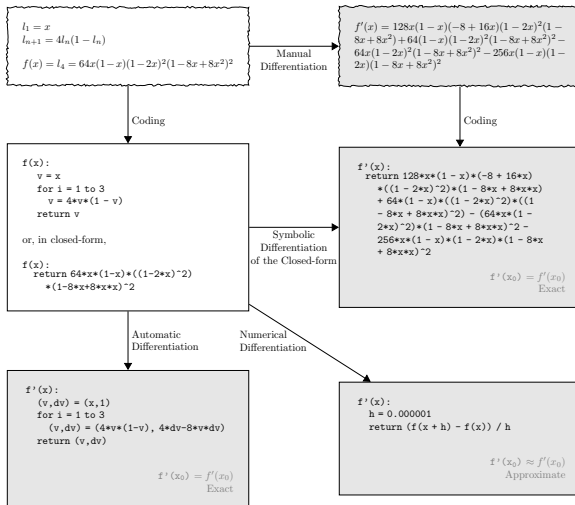
Manual
Differentiation

$$f'(x) = 128x(1-x)(-8+16x)(1-2x)^2(1-8x+8x^2) + 64(1-x)(1-2x)^2(1-8x+8x^2)^2 - 64x(1-2x)^2(1-8x+8x^2)^2 - 256x(1-x)(1-2x)(1-8x+8x^2)^2$$

Coding

```
f'(x):  
return 128*x*(1-x)*(-8+16*x)  
      *((1-2*x)^2)*(1-8*x+8*x*x)  
      + 64*(1-x)*((1-2*x)^2)*((1  
      - 8*x+8*x*x)^2) - (64*x*(1-x)
```

Derivación automática: que no es



Derivación automática: que **no es**

Coding

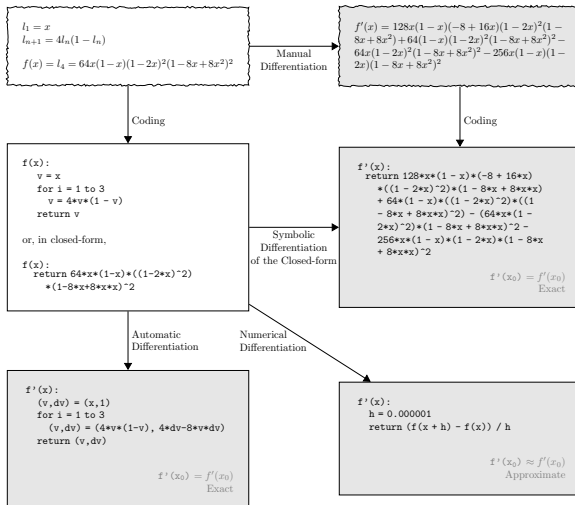
```
f'(x):  
return 128*x*(1 - x)*(-8 + 16*x)  
      *((1 - 2*x)^2)*(1 - 8*x + 8*x*x)  
      + 64*(1 - x)*((1 - 2*x)^2)*((1  
      - 8*x + 8*x*x)^2) - (64*x*(1 -  
      2*x)^2)*(1 - 8*x + 8*x*x)^2 -  
      256*x*(1 - x)*(1 - 2*x)*(1 - 8*x  
      + 8*x*x)^2
```

Symbolic
Differentiation
of the Closed-form

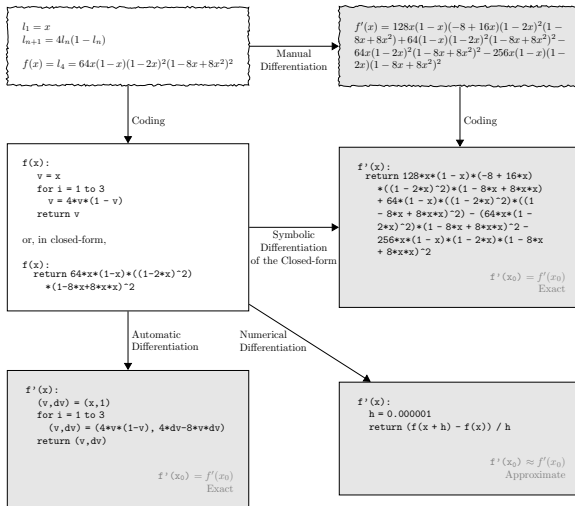
$$f'(x_0) = f'(x_0)$$

Exact

Derivación automática: que no es



Derivación automática



Derivación automática

Automatic
Differentiation

Numerical
Differentiation

```
f'(x):  
  (v,dv) = (x,1)  
  for i = 1 to 3  
    (v,dv) = (4*v*(1-v), 4*dv-8*v*dv)  
  return (v,dv)
```

$$f'(x_0) = f'(x_0)$$

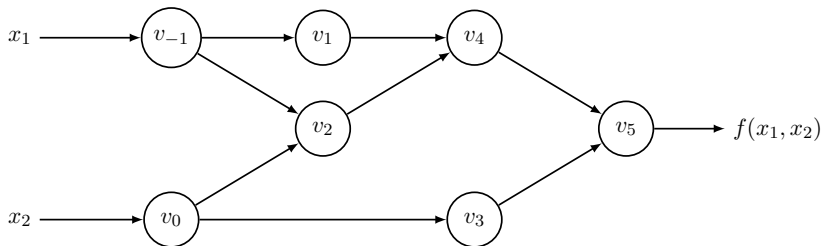
Exact

Derivación automática: otro ejemplo

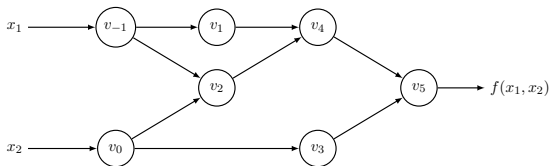
$$f(x_1, x_2) = \ln(x_1) + x_1x_2 - \sin(x_2)$$

Derivación automática: otro ejemplo

$$f(x_1, x_2) = \ln(x_1) + x_1x_2 - \sin(x_2)$$



Derivación automática: modo hacia adelante



$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

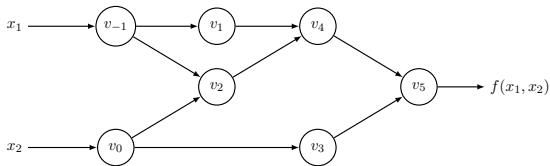
$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

Derivación automática: modo hacia adelante



$$\dot{v}_i = \frac{\partial v_i}{\partial x_1}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

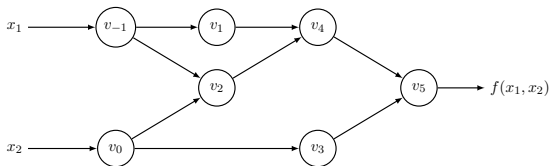
$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

Derivación automática: modo hacia adelante



$$\dot{v}_i = \frac{\partial v_i}{\partial x_1}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

$$\dot{v}_{-1} = \dot{x}_1 = 1$$

$$\dot{v}_0 = \dot{x}_2 = 0$$

$$\dot{v}_1 = \dot{v}_{-1} / v_{-1} = 1/2$$

$$\dot{v}_2 = \dot{v}_{-1} \times v_0 + \dot{v}_0 \times v_{-1} = 1 \times 5 + 0 \times 2$$

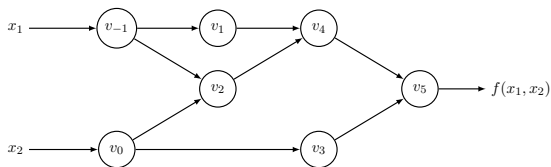
$$\dot{v}_3 = \dot{v}_0 \times \cos v_0 = 0 \times \cos 5$$

$$\dot{v}_4 = \dot{v}_1 + \dot{v}_2 = 0.5 + 5$$

$$\dot{v}_5 = \dot{v}_4 - \dot{v}_3 = 5.5 - 0$$

$$\dot{y} = \dot{v}_5 = 5.5$$

Derivación automática: modo **reverso**



$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

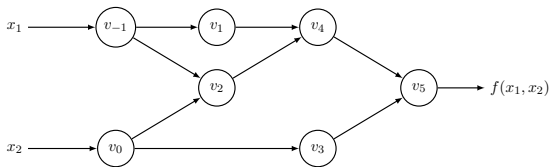
$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

Derivación automática: modo **reverso**



$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

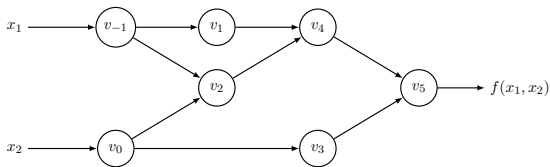
$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

Derivación automática: modo **reverso**



$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

$$v_3 = \sin v_0 = \sin 5$$

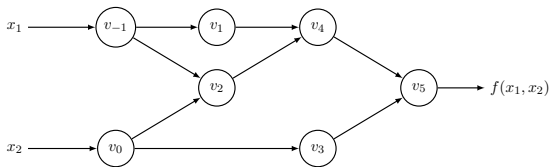
$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

$$\bar{v}_5 = \bar{y} = 1$$

Derivación automática: modo **reverso**



$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

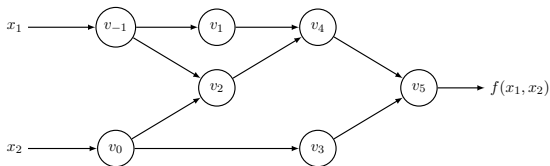
$$y = v_5 = 11.652$$

$$\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3} = \bar{v}_5 \times (-1) = -1$$

$$\bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4} = \bar{v}_5 \times 1 = 1$$

$$\bar{v}_5 = \bar{y} = 1$$

Derivación automática: modo **reverso**



$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

$$\bar{v}_2 = \bar{v}_4 \frac{\partial v_4}{\partial v_2} = \bar{v}_4 \times 1 = 1$$

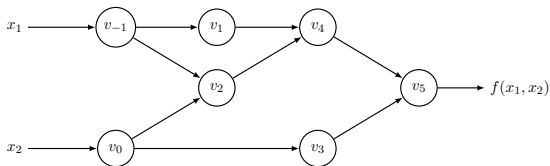
$$\bar{v}_1 = \bar{v}_4 \frac{\partial v_4}{\partial v_1} = \bar{v}_4 \times 1 = 1$$

$$\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3} = \bar{v}_5 \times (-1) = -1$$

$$\bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4} = \bar{v}_5 \times 1 = 1$$

$$\bar{v}_5 = \bar{y} = 1$$

Derivación automática: modo reverso



$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

$$\bar{v}_{-1} = \bar{v}_{-1} + \bar{v}_1 \frac{\partial v_1}{\partial v_{-1}} = \bar{v}_{-1} + \bar{v}_1 / v_{-1} = 5.5$$

$$\bar{v}_0 = \bar{v}_0 + \bar{v}_2 \frac{\partial v_2}{\partial v_0} = \bar{v}_0 + \bar{v}_2 \times v_{-1} = 1.716$$

$$\bar{v}_{-1} = \bar{v}_2 \frac{\partial v_2}{\partial v_{-1}} = \bar{v}_2 \times v_0 = 5$$

$$\bar{v}_0 = \bar{v}_3 \frac{\partial v_3}{\partial v_0} = \bar{v}_3 \times \cos v_0 = -0.284$$

$$\bar{v}_2 = \bar{v}_4 \frac{\partial v_4}{\partial v_2} = \bar{v}_4 \times 1 = 1$$

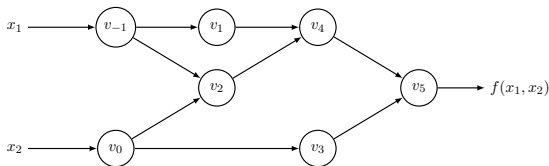
$$\bar{v}_1 = \bar{v}_4 \frac{\partial v_4}{\partial v_1} = \bar{v}_4 \times 1 = 1$$

$$\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3} = \bar{v}_5 \times (-1) = -1$$

$$\bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4} = \bar{v}_5 \times 1 = 1$$

$$\bar{v}_5 = \bar{y} = 1$$

Derivación automática: modo reverso



$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

$$\bar{x}_1 = \bar{v}_{-1}$$

$$\bar{x}_2 = \bar{v}_0$$

$$= 5.5$$

$$= 1.716$$

$$\bar{v}_{-1} = \bar{v}_{-1} + \bar{v}_1 \frac{\partial v_1}{\partial v_{-1}} = \bar{v}_{-1} + \bar{v}_1 / v_{-1} = 5.5$$

$$\bar{v}_0 = \bar{v}_0 + \bar{v}_2 \frac{\partial v_2}{\partial v_0} = \bar{v}_0 + \bar{v}_2 \times v_{-1} = 1.716$$

$$\bar{v}_{-1} = \bar{v}_2 \frac{\partial v_2}{\partial v_{-1}} = \bar{v}_2 \times v_0 = 5$$

$$\bar{v}_0 = \bar{v}_3 \frac{\partial v_3}{\partial v_0} = \bar{v}_3 \times \cos v_0 = -0.284$$

$$\bar{v}_2 = \bar{v}_4 \frac{\partial v_4}{\partial v_2} = \bar{v}_4 \times 1 = 1$$

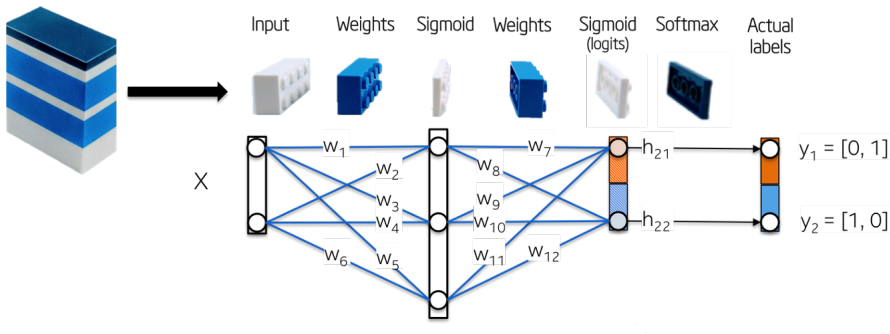
$$\bar{v}_1 = \bar{v}_4 \frac{\partial v_4}{\partial v_1} = \bar{v}_4 \times 1 = 1$$

$$\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3} = \bar{v}_5 \times (-1) = -1$$

$$\bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4} = \bar{v}_5 \times 1 = 1$$

$$\bar{v}_5 = \bar{y} = 1$$

Programación diferenciable: el Lego de las arquitecturas neuronales profundas



Introducción
○○○

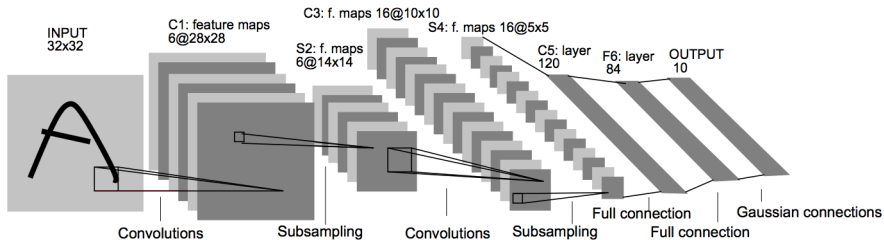
Una neurona
○○○○○○○○○○○○

Varias neuronas
○○○○○○○○○○○○○○○○○○○○○○

Muuuuuuuuuuuchas neuronas
○○○○○○○○○○○○○○●○○○○○○○○○○

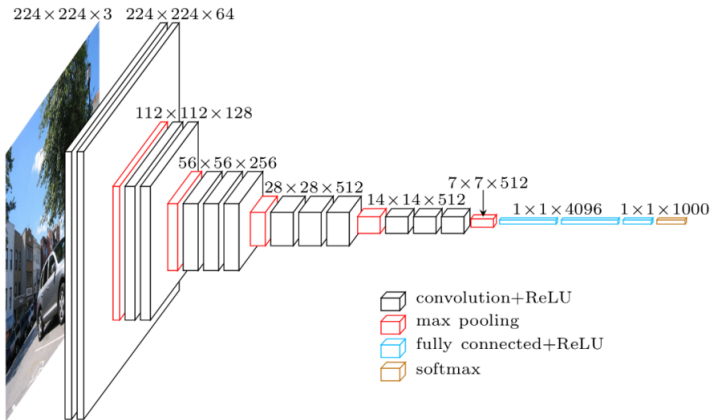
$$\partial P \equiv$$


LeNet-5



$$\partial P \equiv \text{blue brick}$$

VGG-16



Introducción
○○○

Una neurona
○○○○○○○○○○○○

Varias neuronas
○○○○○○○○○○○○○○○○○○○○○○

Muuuuuuuuuuuuuchas neuronas
○○○○○○○○○○○○○○○○○○●○○○○○○○○

$$\partial P \equiv \text{blue brick}$$

Inception

convolution

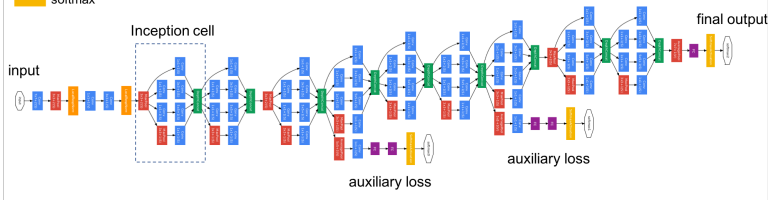
max pooling

channel concatenation

channel-wise normalization

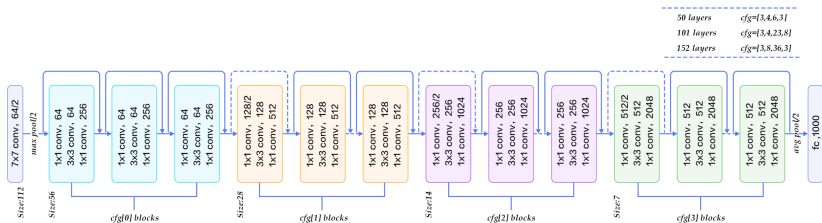
fully-connected layer

softmax



$$\partial P \equiv \text{blue brick}$$

ResNet



Introducción
○○○

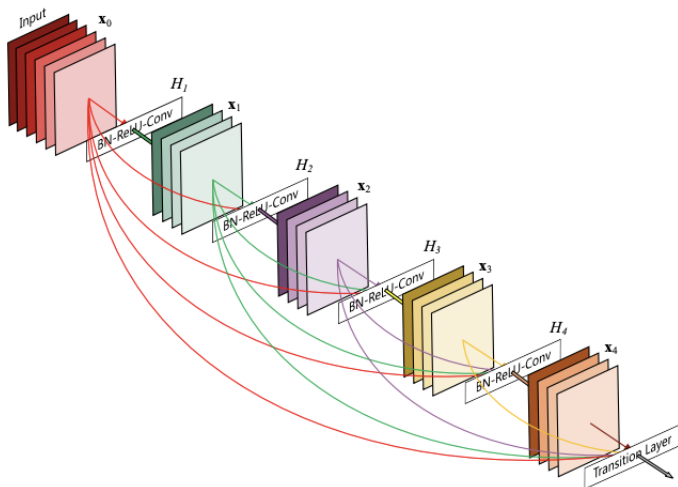
Una neurona
○○○○○○○○○○○○○○

Varias neuronas
○○○○○○○○○○○○○○○○○○○○○○○○○○○○

Muuuuuuuuuuuuuchas neuronas
○○○○○○○○○○○○○○○○○○○○●○○○○○○○○

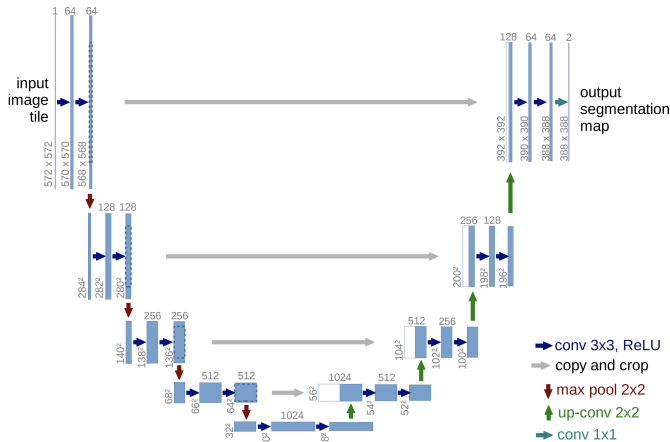
$$\partial P \equiv \text{blue brick}$$

DenseNet



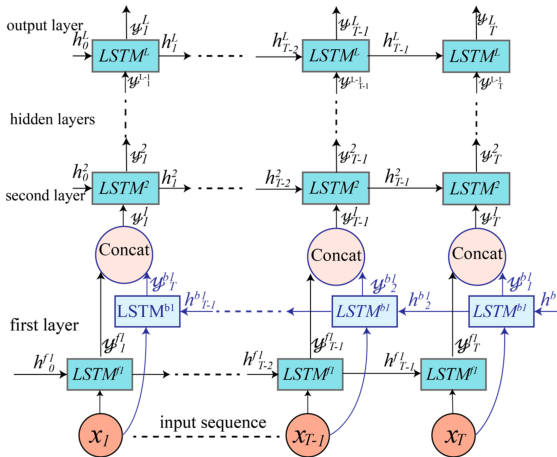
$$\partial P \equiv$$


U-Net



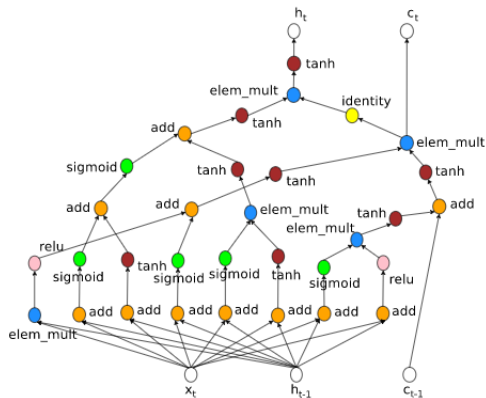
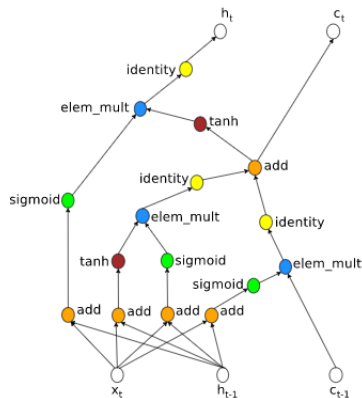
$$\partial P \equiv \text{blue brick}$$

LSTM



$$\partial P \equiv \text{brick}$$

Diseños automáticos



Arquitecturas neuronales tradicionales

- Redes de propagación hacia adelante (feedforward)
- Redes recurrentes (recurrent/with feedback)
- Otros modelos híbridos

Arquitecturas neuronales tradicionales

- Redes de propagación hacia adelante (feedforward)
 - Perceptrones simples y de una capa
 - Perceptrón multicapa
 - Redes con funciones de base radial
 - ...
- Redes recurrentes (recurrent/with feedback)
- Otros modelos híbridos

Arquitecturas neuronales tradicionales

- Redes de propagación hacia adelante (feedforward)
 - Perceptrones simples y de una capa
 - Perceptrón multicapa
 - Redes con funciones de base radial
 - ...
- Redes recurrentes (recurrent/with feedback)
 - Redes de Kohonen
 - Redes de Hopfield
 - Redes de Boltzmann
 - Redes de Elman/Jordan
 - Modelos de resonancia adaptativa
 - ...
- Otros modelos híbridos

Arquitecturas neuronales tradicionales

- Redes de propagación hacia adelante (feedforward)
 - Perceptrones simples y de una capa
 - Perceptrón multicapa
 - Redes con funciones de base radial
 - ...
- Redes recurrentes (recurrent/with feedback)
 - Redes de Kohonen
 - Redes de Hopfield
 - Redes de Boltzmann
 - Redes de Elman/Jordan
 - Modelos de resonancia adaptativa
 - ...
- Otros modelos híbridos
 - Parcialmente recurrentes
 - Parcialmente conectados
 - Redes modulares
 - ...

Arquitecturas profundas

- Redes de propagación hacia adelante (feedforward)
- Redes recurrentes (recurrent/with feedback)
- Otros modelos y métodos de entrenamiento

Arquitecturas profundas

- Redes de propagación hacia adelante (feedforward)
 - Deep Belief Network (DBN)
 - Convolutional Neural Network (CNN)
 - ... LeNet, AlexNet, ZF Net, GoogleLeNet, VGGNet, U-Net, ResNet, DenseNet, CapsuleNet, GCN...
 - ...
- Redes recurrentes (recurrent/with feedback)
- Otros modelos y métodos de entrenamiento

Arquitecturas profundas

- Redes de propagación hacia adelante (feedforward)
 - Deep Belief Network (DBN)
 - Convolutional Neural Network (CNN)
 - ... LeNet, AlexNet, ZF Net, GoogleLeNet, VGGNet, U-Net, ResNet, DenseNet, CapsuleNet, GCN...
 - ...
- Redes recurrentes (recurrent/with feedback)
 - Long Short-Term Memory (LSTM)
 - Attention Mechanisms
 - Transformers
 - ...
- Otros modelos y métodos de entrenamiento

Arquitecturas profundas

- Redes de propagación hacia adelante (feedforward)
 - Deep Belief Network (DBN)
 - Convolutional Neural Network (CNN)
 - ... LeNet, AlexNet, ZF Net, GoogleLeNet, VGGNet, U-Net, ResNet, DenseNet, CapsuleNet, GCN...
 - ...
- Redes recurrentes (recurrent/with feedback)
 - Long Short-Term Memory (LSTM)
 - Attention Mechanisms
 - Transformers
 - ...
- Otros modelos y métodos de entrenamiento
 - Generative Adversarial Networks (GAN)
 - Deep Reinforcement Learning (DRL)
 - Bayesian Deep Learning (BDL)
 - ...

Procesos de aprendizaje

- Tipos de aprendizaje
- Reglas de aprendizaje

Procesos de aprendizaje

- Tipos de aprendizaje
 - Supervisado
 - No supervisado
 - Por refuerzo
 - Híbridos
- Reglas de aprendizaje

Procesos de aprendizaje

- Tipos de aprendizaje
 - Supervisado
 - No supervisado
 - Por refuerzo
 - Híbridos
- Reglas de aprendizaje
 - Corrección de error
 - Competitivo
 - Hebbiano
 - Boltzman

Procesos de aprendizaje

- Tipos de aprendizaje
 - Supervisado
 - No supervisado
 - Por refuerzo
 - Híbridos
- Reglas de aprendizaje
 - Corrección de error
 - Competitivo
 - Hebbiano
 - Boltzman
- Y muchos más...
 - semi-supervised, self-supervised, transductive,
 - incremental, adaptive, online, active, multi-label, multi-task,
 - multi-instance, zero-shot, one-shot, one-class, feature
 - leraning, transfer lerarning, generative adversarial
 - learning,...

Gracias por su atención!

sinc*(i)*

Instituto de Investigación en Señales,
Sistemas e Inteligencia Computacional

Diego Milone

`dmilone@sinc.unl.edu.ar`



s i n c (i)

`www.sinc.unl.edu.ar`

Departamento de Informática
FICH - Universidad Nacional del Litoral
Consejo Nacional de Investigaciones Científicas y Técnicas
Ciudad Universitaria UNL - 3000 - Santa Fe - Argentina